



LineFlowDP: A Deep Learning-Based Two-Phase Approach for Line-Level Defect Prediction

Fengyu Yang^{1,2} · Fa Zhong^{1,2} · Guangdong Zeng^{1,2} · Peng Xiao^{1,2} · Wei Zheng^{1,2}

Accepted: 17 December 2023 / Published online: 23 February 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Software defect prediction plays a key role in guiding resource allocation for software testing. However, previous defect prediction studies still have some limitations: (1) the granularity of defect prediction is still coarse, so high-risk code statements cannot be accurately located; (2) in fine-grained defect prediction, the semantic and structural information available in a single line of code is limited, and the content of code semantic information is not sufficient to achieve semantic differentiation. To address the above problems, we propose a two-phase line-level defect prediction method based on deep learning called LineFlowDP. We first extract the program dependency graph (PDG) of the source files. The lines of code corresponding to the nodes in the PDG are extended semantically with data flow and control flow information and embedded as nodes, and the model is further trained using an relational graph convolutional network. Finally, a graph interpreter GNNExplainer and a social network analysis method are used to rank the lines of code in the defective file according to risk. On 32 datasets from 9 projects, the experimental results show that LineFlowDP is 13%-404% more cost-effective than four state-of-the-art line-level defect prediction methods. The effectiveness of the flow information extension and code line risk ranking methods was also verified via ablation experiments.

Keywords Software quality assurance · Line-level software defect prediction · Graph neural network · Social network analysis

1 Introduction

With the continuous development of software engineering technology, the study of software defects has gradually become a hotspot of software reliability research (Wong et al., 2010; Wong et al., 2017). According to IEEE Standard 1044-2009(Ieee, 1994), a

Communicated by: Leandro L. Minku

✉ Fa Zhong
zhongfa1999@qq.com

¹ School of Software, Nanchang Hangkong University, Nanchang 330063, People's Republic of China

² Software Testing and Evaluation Center, Nanchang Hangkong University, Nanchang 330063, People's Republic of China

software defect is “an imperfection or defect in a software product that does not meet its requirements or specifications and needs to be repaired or replaced”. Currently, many researchers are using machine learning (ML) techniques and deep learning (DL) techniques for software defect prediction (SDP). In addition, the granularity of defect prediction is mainly divided into the file level, class level and method level (in projects implemented in object-oriented programs, file granularity is also called class granularity(Chen et al., 2019)). However, the prediction granularity of most current methods is still at the file level, which is considered coarse-grained(Wan et al., 2018a). According to statistics, only 1%-3% of the lines of code in a file are defective(Wattanakriengkrai et al., 2020): this indicates that when a file is predicted to be defective, developers may waste considerable time and effort in locating high-risk lines of code in the file. Line-level SDP models can help developers and testers quickly locate high-risk lines of code, thus significantly reducing the cost of software defect repair(Pornprasit & Tantithamthavorn, 2022). However, current line-level SDP methods still have some limitations.

- (1) **A single line of code provides limited semantic and structural information for line-level defect prediction.** In the coarse-grained software defect prediction(Yang et al., 2023; Zheng et al., 2023; Wang et al., 2018; Jian et al., 2017; Dam et al., 2018), researchers treated source code as text(Li et al., 2022). Because source code contains sufficient semantics of itself, extracting semantic information using natural language processing methods tends to consider only the current granularity of statements without considering surrounding statements. However, when implementing line-level defect prediction, the content of semantic information contained in a single line of code is not semantically distinguishable due to the lack of semantics(Li et al., 2022), a line of code is not enough to reflect the semantics and complexity it contains(Hindle et al., 2008; Kondo et al., 2020) and a line of code provides very limited information about whether the line is defective or not.
- (2) **Limitations of model score methods and post-hoc interpretation methods.** In previous line-level software defect prediction methods, the risk of a line of code was judged via model score mechanisms(Pornprasit & Tantithamthavorn, 2022; Wang et al., 2016) (e.g., attention mechanism or cross-entropy), and the higher the model score of a line of code, the more it was considered to contribute to the defect. These methods have achieved good results in defect prediction. However, there are limitations to the use of model scores, as these models include statements that are not directly involved in code execution (e.g., block terminators such as “}”, “package” statements, “import” statements, “else” statements, etc.) in the model training. Notably, these non-executable statements are unlikely to introduce post-release defects because there are no data or control dependency relationships between them and other statements. In contrast to the model score mechanism-based approach, another fine-grained SDP method called the post-hoc explanation-based approach, was proposed by Li et al.(Li et al., 2021), who constructed a program dependence graph (PDG) in which only statements directly involved in code execution were retained and then used an improved graph neural network (GNN) to learn structural and semantic information. A fine-grained interpretation can be extracted using the post-hoc interpretation method GNNExplainer(Ying et al., 2019) for the files predicted to be defective, which results in a key subgraph containing

the key statements associated with the defect; however, the granularity of the results generated by this method is still coarse and does not achieve line-level granularity.

In this paper, we mainly focus on the above limitations and propose LineFlowDP while considering using the data flow and control flow information in PDG for single line code semantics for predicting whether files are defective and ranking the risk of code lines. The main contributions of this paper are as follows:

- (1) We propose using the flow information in the PDG to extend the semantics of a single line of code, that is, to extend the original line of code into a code statement composed of backwards data flow, backwards control flow, the current line of code, forwards control flow and forwards data flow. This effectively solves the problem that the semantic information of a single line of code is not enough to be distinguishable.
- (2) We propose a deep learning-based two-phase approach for line-level defect prediction, LineFlowDP. By constructing a PDG of source code and using flow information to conduct semantic extension of code lines, we train an relational graph convolutional network (RGCN) model to predict whether there are defects in the file and use the graph interpreter GNNExplainer and social network analysis (SNA) to sort the lines of code in the defective file by risk, which provides a novel method for line-level defect prediction.
- (3) We evaluate the effectiveness of LineFlowDP on 32 versions of 9 projects using 4 traditional evaluation metrics (AUC-ROC, Balanced Accuracy, Matthews Correlation Coefficient, Grometric Mean between TPR and TNR) for file-level defect prediction, 4 evaluation metrics (Recall@Top20%LOC, Effort@Top20%Recall, Initial False Alarms and Top-k Accuracy) for line-level defect prediction and verify the contribution of each component of LineFlowDP by ablation experiments.
- (4) To foster the replication of our study, we published the implementation of our LineFlowDP and the baselines at <https://github.com/LineFlowDP/LineFlowDP>.

The paper is organized as follows: Section 1 is the introduction to this paper, Section 2 introduces the tools used in the paper and the research and work related to SDP, Section 3 details the LineFlowDP methodological framework, Section 4 describes the experimental design, Section 5 provides the experimental results and analysis, Section 6 contains the discussion, Section 7 describes the validity threats, and Section 8 provides a brief summary of the paper.

2 Related Work

2.1 Program Dependency Graphs

A PDG is an intermediate program representation that specifies the data and control dependencies for each operation in the program (Ferrante et al., 1984). Data dependency is used to represent the data flow relationship in the program, reflecting the interactions between variables; control dependency represents the control flow relationship in the program, which is derived from the traditional control flow diagram and can be used to

reflect the execution of code. Due to the rich source code structure information contained in PDGs, they have been widely used in defect prediction and related fields (such as vulnerability detection (Li et al., 2021; Wu et al., 2022b) and fault localization (Xiao et al., 2017)). Korel et al. (Korel, 1987) proposed a method based on PDGs to detect redundant code by representing the program structure in terms of data dependency and control dependency between program instructions. Xiao et al. (Xiao et al., 2017) proposed a new probabilistic graph model based on PDGs and Bayesian networks, which has excellent ability to reason across nonadjacent nodes and can more accurately detect faults. Cao et al. (Cao et al., 2022) achieved higher detection accuracy on memory-related vulnerabilities by using an optimized GNN to learn the structural information in a PDG. The data and control dependencies contained in the PDG can provide important information for defect prediction.

2.2 Defect Prediction Method based on GNNs

A GNN is a general algorithm that uses neural networks to learn graph-structured data, extract and mine features and patterns in graph-structured data, and perform graph learning tasks such as clustering, classification, prediction, segmentation, and generation (Wu et al., 2022a). The powerful graph structure learning function of GNNs enables them to achieve good prediction results in defect prediction. Zeng et al. (Zeng et al., 2021) obtained the structural features of a network by constructing a class dependency network using node-2vec embedded learning, combining the learned structural features with traditional software code features for initializing the node attributes in the class dependency network, and further feeding the dependency network into a graph convolution neural network (GCN) model for training. The experimental results show that their proposed method outperforms the state-of-the-art defect prediction methods. Xu et al. (Xu et al., 2022) created an improved defect representation and software prediction model, proposed a defect region candidate method based on augmented code property graphs (CPGs) to extract the defect category associated with the defect region, learned the defect features and predicted the defect by using a GNN. The experimental results show that the method can effectively predict specific types of defects. In response to GNN-based defect prediction methods that ignore the importance of traditional manual features, Tang et al. (Tang et al., 2022) proposed building graphs by connecting parent and child nodes of Abstract Syntax Trees (ASTs) and connecting leaf nodes from left to right to preserve the contextual information contained in the source code. The semantic and structural features of the source code are then automatically generated using GCN, and the features learned by GCN are combined with manual features to obtain a more effective prediction model. Many studies have shown the excellent performance of GNNs in defect prediction, so GNNs are also used as a defect prediction model in our study.

2.3 File-level and Line-level Defect Prediction

File-level granularity is one of the most common types of granularity in software defect prediction. Researchers have proposed numerous methods to improve defect prediction performance at the file level. To address the traditional SDP methods often ignoring the semantic differences between programs, Wang et al. (Wang et al., 2018; Jian et al., 2017) use an abstract syntax tree (AST) from programs to automatically learn semantic features using deep learning techniques. In addition, Dam et al. (Dam et al., 2018) used a long short-term memory (LSTM) architecture to learn the semantic and syntactic features of

source code. The experimental results show that semantic features have significant advantages over traditional features for file-level SDP. Recently, pre-trained models for natural languages (NL) have been recently shown to transfer well to programming languages and largely benefit a broad set of code-related tasks. CodeT5(Wang et al., 2021) is a unified pre-trained encoder-decoder Transformer model that better leverages the code semantics conveyed from the developer-assigned identifiers. It employs a unified framework to seamlessly support both code understanding and generation tasks and allows for multi-task learning. CodeT5 also shows impressive performance in defect detection.

Since the granularity of most defect prediction approaches is at the file level, when a defect is predicted in a file, further code review is required to determine the exact location of the defect, which consumes developer time and effort. Most existing work in large-scale software defect prediction focuses on determining whether a source file is likely to be defective or not. This level of granularity has become the standard in the literature of software defect prediction(Kamei & Shihab, 2016). Therefore, researchers have proposed line-level software defect prediction to quickly locate the location of defects.

Li et al. (Li et al., 2021) proposed a vulnerability detection method, IVDetect, which is more fine-grained. The first step in this method is to extract the program dependence graph (PDG) of the source code. Next, a GNN model is used to predict whether vulnerabilities exist in the PDG. Finally, an interpretable artificial intelligence method is employed. This method extracts a fine-grained explanation, which includes subgraphs in the PDG and key statements that are related to the vulnerability. However, the fine-grained interpretation subgraph extracted by this method still contains many statements and does not have line-level granularity. Wattanakriengkrai et al.(Wattanakriengkrai et al., 2020) proposed using a model-agnostic interpretation approach to achieve line-level defect prediction. The authors first used code token features to build a file-level defect model and then used a local interpretable model-agnostic explanations (LIME)(Ribeiro et al., 2016) to interpret the prediction results of file defects, identifying the tokens that caused the file to be predicted as defective and the lines of code containing risky tokens as risky lines. To fully utilize the surrounding tokens and surrounding lines, Pornprasit et al.(Pornprasit & Tantithamthavorn, 2022) proposed a deep learning-based line-level defect prediction method for this problem, first generating a vector representation for the code token and then learning the neighbouring tokens using a gated recurrent unit (GRU)(Cho et al., 2014) in both directions. Furthermore, hierarchical attention networks (HANs)(Yang et al., 2016) were used to learn the hierarchical structure of the source code (e.g., tokens constitute a line, lines constitute a file) to predict whether the file was defective or not. Finally, the attention mechanism(Vaswani et al., 2017) was used to calculate the risk score of the code tokens in the defective file, and the sum of the risk scores of the tokens in a line of code was used as the risk score of the line of code. Experiments show that the risk scores of code tokens calculated by this method vary greatly depending on their location and that this method is more cost-effective than other line-level defect prediction methods. The above research provides a new idea for line-level SDP and motivates us to propose a new method for line-level SDP.

2.4 Explainable AI for Software Engineering

Although ML techniques are used in numerous studies, most ML predictions are a black box, and it is not clear how they work. To generate interpretable and locally plausible model explanations, Ribeiro et al.(Ribeiro et al., 2016) proposed local interpretable

model-agnostic explanations (LIME), LIME is a widely used technique for interpreting black box machine learning models by constructing interpretable local models. It generates similar but slightly different samples by making small perturbations or sampling the instances to be interpreted. The interpretable model is then trained to simulate the predictive behaviour of the black-box model. By calculating feature weights, LIME identifies which features have the greatest impact on the prediction results. It is applicable to a wide range of ML models and provides flexible parameter settings to meet different interpretation needs. LIME is also used in SDP. For example, Pornprasit et al.(Pornprasit & Tantithamthavorn, 2021) utilized LIME for line-level just-in-time defect prediction. Wattanakriengkrai et al.(Wattanakriengkrai et al., 2020) used model-agnostic techniques (e.g., LIME) to explain defect prediction models and help developers better understand why files are predicted to be defective. In addition, many model-agnostic methods have been proposed for interpreting models, such as SHAP(Lundberg & Lee, 2017), and breakDown(Staniak & Biecek, 2018).

Graph neural networks are a powerful tool for ML on graphs. Due to the complexity of graph-structured data, interpreting the predictions made by GNNs remains unsolved. Therefore, Ying et al.(Ying et al., 2019) proposed GNNExplainer, a model-agnostic approach for providing interpretable explanations for predictions of any GNN-based model on any graph-based machine learning task. When an instance is given, GNNExplainer identifies a compact subgraph structure and a small subset of node features that have a crucial role in GNN prediction. The emergence of GNNExplainer can help researchers understand and dig deeper into the internal mechanisms of graph neural networks, especially in the fields of social network analysis, chemical molecular structure prediction, and recommender systems, which are widely used. In addition, some other graph interpreters have been proposed, such as Cf-gnnexplainer(Lucic et al., 2022) and KS-GNNExplainer(Abdous et al., 2023).

3 Methodology

In this paper, we propose a two-phase line-level software defect prediction method called LineFlowDP. Our method addresses the lack of semantic information of a single line of code and the coarse granularity of prediction. In this section, the proposed LineFlowDP method is described. The flow consists of three main steps: (0) Data Preparation; (1) Phase 1: File-level prediction phase; (2) Phase 2: Line-level prediction phase.

The LineFlowDP framework is shown in Fig. 1. (0) Data preparation. First, the source code of the PDG is extracted, and its nodes and edges are refined to retain as much structural information as possible; each line of code corresponding to a node in the PDG is semantically extended using data flow and control flow information, the extended statement is converted into a low-dimensional space vector representation using the Doc2Vec(Le & Mikolov, 2014) word vector technique, and the converted vector is used as the node embedding of the PDG. (1) Phase 1: File-level prediction phase. An RGCN model is trained and saved to predict whether the file is defective or clean. (2) Phase 2: Line-level prediction phase. If the file is defective, the RGCN model and the corresponding PDG are fed into the graph interpreter GNNExplainer, which generates a minimal subgraph that affects the model results. Then we use SNA methods to calculate the importance of the nodes in the minimal subgraph, and use the node importance as the code line risk score to finally achieve the code line risk ranking.

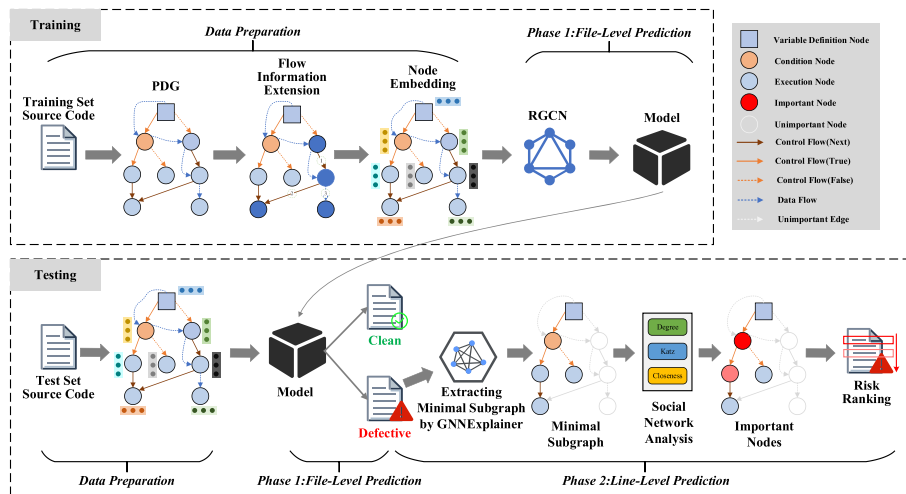


Fig. 1 Framework of LineFlowDP

3.1 Data Preparation

We apply the method of Hoang H et al.(Nguyen et al., 2022) to extract the PDG and further refine the nodes and control flow edges in the traditional PDG. First, the PDG is extracted based on the source code, which contains nodes, control flow edges and data flow edges, and we further refine the nodes and edges of the traditional PDG, as shown in Fig. 2. In the original PDG, all lines of code are considered to be nodes of the same type, but in the refined PDG, the nodes can be one of three types: variable definition, condition or execution nodes. The control flow edges are also one of three types: edges executed by a True statement, edges executed by a False statement and edges executed Next time. Refined nodes can distinguish different types of lines of code, and refined edges can reflect the execution flow of source code more clearly. By further refining the original PDG, we can make full use of the structural information of the source code and explore the defect-related information in the PDG structure.

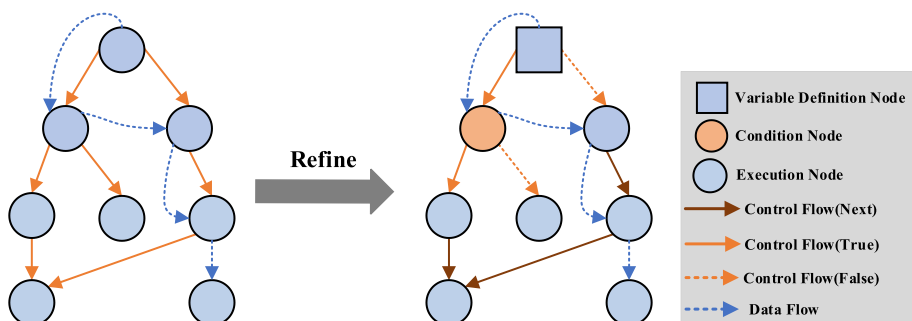


Fig. 2 Refined PDG schematic

3.1.1 Extending Code Line Semantics using Flow Information in PDG

Before node embedding, in line-level defect prediction, the semantic and structural information available in a single line of code is very limited. Due to insufficient semantic information of the code, it is impossible to achieve semantic differentiation, and thus, the model learns fewer defect-related features. If we use the context of the neighbouring lines of the code as additional information about the current line, the execution order of the code and the use of variables will be ignored, and the execution order of the code may not be top-down (e.g., after the judgement statement, only the branch statement with a value of True will be executed), so only the context information may not reflect the execution flow of the code, resulting in the loss of some semantic information or redundant information. Therefore, we propose using the data flow and control flow information in the PDG to extend the semantics of the current code line.

The code in Fig. 3 is an example snippet of the Java method in ActiveMQ. The corresponding PDG is also shown. The defective line of code is in the red box in line 116. Even though the logical exception is caught by adding try and catch statements in lines 109 and 120-122, the code may have an effect on other statements and thus introduce a post-release defect.

LineFlowDP extends the semantics of code lines in the following order: backwards data flow→backwards control flow→current code line→forwards control flow→forwards data flow. Considering the importance of code readability, programmers used to place code statements within the True branch in default, so we select the priority of the extended control flow as follows: control flow (True)>control flow (Next)>control flow (False). In this paper, we consider the flow information extension, which is more in line with the code execution process and to a certain extent ensures line-level granularity. The diagram of flow information extension for the 116th line of the defective code statement is shown in Fig. 3. The backwards data flow of the 116th line of code is 108 lines, the backwards control flow is 115 lines, the forwards control flow is 117 lines, and the forwards data flow is 118 lines, so the extended code statements are 108, 115, 116, 117, and 118, as shown in Fig. 4.

Kondo et al. (Kondo et al., 2020) showed that contextual information has an important impact on the semantics of the current line of code, so we similarly use contextual information to extend the semantics of a single line of code. Fig. 5 shows an example of extending

● Extending Code Line Semantics using Flow Information in PDG

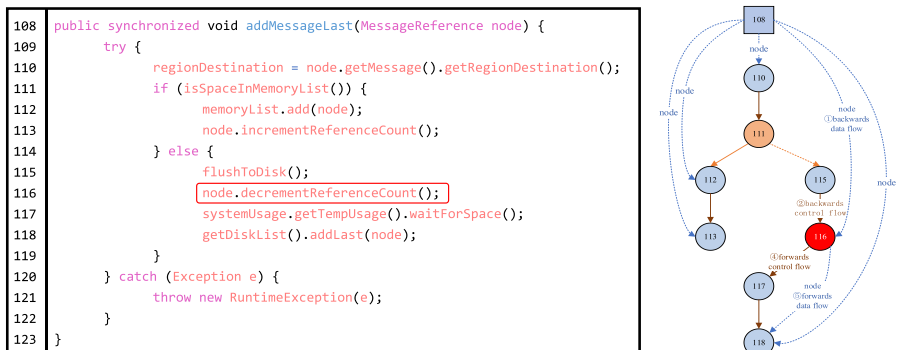


Fig. 3 Code example and corresponding PDG

Fig. 4 Code statements after flow information extension

```

108 public synchronized void addMessageLast(MessageReference node) {
115     flushToDisk();
116     node.decrementReferenceCount();
117     systemUsage.getTempUsage().waitForSpace();
118     getDiskList().addLast(node);

```

Fig. 5 Simple contextual information extension

```

114 } else {
115     flushToDisk();
116     node.decrementReferenceCount();
117     systemUsage.getTempUsage().waitForSpace();
118     getDiskList().addLast(node);

```

information by using two neighbouring statements above and below the current line of code. The simple contextual information extension captures 114 lines of code that do not have direct data or control dependencies with the defective line 116, which is likely to lead to semantic redundancy, while the 108 lines of code obtained by flow information extension in Fig. 4 contain the variable definition of “node” in line 116. Compared with the contextual nearby information extension, the remaining code statements selected by the flow information extension have data or control dependencies with the original code because the flow information is derived from the PDG of the source code, so the extended statements are more closely related to the current statements in terms of semantics and structure.

3.1.2 Node Embedding

Node embedding converts the extended code lines into a low-dimensional space vector representation as nodes in the PDG, which contains the semantic information of the code statements. The node-embedded PDG is then used as the input of the RGCN model.

Doc2Vec(Le & Mikolov, 2014) is an unsupervised technique for converting text to vectors, which has been used and accepted by a wide range of researchers. Compared with other word embedding methods that crop overly long tokens in the process of obtaining a fixed-length vector, Doc2Vec has the advantage of being able to represent paragraph/document features more completely and accurately(Cheng et al., 2021). Unlike Word2vec, which converts words into vectors, Doc2Vec can convert arbitrary variable-length text into fixed-length vectors. Therefore, in this paper, we use Doc2Vec to convert extended lines of code into vectors as node embeddings of graphs. Doc2Vec converts variable-length text into fixed-length vectors roughly as follows.

Given a corpus consisting of token sequences $w_1, w_2, \dots, w_n$, including code tokens and code line statements, our task is to predict token w_i based on the surrounding token sequences $w_{i-k}, \dots, w_{i+k}$ within a window of size $2k+1$ (or fixed-length context). When the average log probability of tokens is maximized, tokens with similar meanings are more similar in distance in the potential vector space:

$$\frac{1}{N} \sum_{i=k}^{N-k} \log p(w_i | w_{i-k}, \dots, w_{i+k}) \quad (1)$$

The probability of w_i is usually calculated with the *softmax* multiclassifier:

$$p(w_i | w_{i-k}, \dots, w_{i+k}) = \frac{\exp(y_{w_i})}{\sum_j \exp(y_{w_j})} \quad (2)$$

where y_{w_i} represents the i th element in the vector of nonnormalized log probability distributions of the output token.

$$\bar{y} = \mathbf{U}h(w_{i-k}, \dots, w_{i+k}; \mathbf{V}) + b \quad (3)$$

where each column of $\mathbf{V}$ represents a unique vector of each token (i.e., code token and line of code) and is randomly initialized, and $\mathbf{U}$ and b are the *softmax* parameters. $h(\cdot)$ denotes the token vector extracted from splice $\mathbf{V}$.

Then, the representation vectors of code tokens are trained and automatically updated using stochastic gradient descent (SGD)(Robbins & Monro, 1951). After training, each code token vector is considered a feature of the token, while the code line vector contains the semantic information of the whole code statement.

After training Doc2Vec on the corpus of each project, the semantic information of the entire extended code statements can be obtained to represent the corresponding nodes in the PDG.

3.2 Phase 1: File-Level Prediction Phase

After the training step to extract the PDG corresponding to the source code, the lines of code in the PDG are semantically extended using flow information, and node embedding is performed. Furthermore, the classification task is implemented by constructing an RGCN model for the PDG to predict the presence of defects in the file.

GCN extends convolution operations from traditional data (e.g., images) to graph data(Wu et al., 2020). The key idea of GCN is to learn the function f_i ; it generates a representation of node v_i by aggregating its own features x_i and the features x_j of its neighbours, where $j \in N(v_i)$ and $N(v_i)$ denotes all neighbouring nodes of node v_i (Wu et al., 2020). GCNs have been effective in several data-driven software engineering tasks (e.g., code representation(Allamanis et al., 2017), clone detection(Wang et al., 2020), and fault location(Lou et al., 2021)).

Although GCN has achieved good results in the above research areas, to make more full use of the refined PDG, we employ RGCN, which can learn different edge types(Schlichtkrull et al., 2018). RGCN is able to learn different edge types in the graph, as well as learn specific weights for each edge type. By utilizing the relationship and semantic information between different edge types (data flow and refined control flow edges), the RGCN is able to capture the complex relationships in the graph.

The main idea of RGCN is to aggregate the features of neighbouring nodes around node i to compute its representation $h_i^{(l+1)}$. Specifically, for each node i , we aggregate its neighbours with different edge types using the following equation:

$$h_i^{(l+1)} = \sigma \left(\sum_{r=1}^M \frac{1}{c_{i,r}} \sum_{j \in N_i^r} W_r^{(l)} h_j^{(l)} \right) \quad (4)$$

Here, M represents the number of edge types, N_i^r denotes the set of r -th type neighbours of node i , $c_{i,r}$ is the count of r -th type neighbours of node i , $W_r^{(l)}$ is the weight matrix, and $\sigma(\cdot)$ is the non-linear activation function.

After each aggregation round, we obtain a new representation $h^{(l+1)}$ for the graph. By performing multiple rounds of aggregation, RGCN can obtain refined node representations. To generate a graph-level representation, various pooling or aggregation methods such as max-pooling or mean-pooling can be applied to aggregate the node representations.

Finally, the graph-level representation is fed into a classifier to predict the category or label of the entire graph. In this way, RGCN enables graph classification by effectively capturing the intricate relationships and structural patterns within the graph data.

In this paper, we use a global mean pooling to perform a pooling operation on the node features to obtain a feature representation of the entire graph. This feature representation is then passed through a fully connected layer and a *Sigmoid* activation function, which outputs the predicted probability values. This probability is used as a basis for predicting whether a file is defective or not.

3.3 Phase 2: Line-Level Prediction Phase

If the file is predicted to be defective, LineFlowDP further analyses the code line risk. The code line risk ranking method is shown in Fig.6.

LineFlowDP inputs both the PDG corresponding to the defective file and the trained RGCN model into GNNExplainer to obtain the minimal subgraph that affects the model results and further extracts the results at a finer granularity. GNNExplainer minimizes the difference in scores between the RGCN model's prediction of the minimal subgraph and the entire PDG by adding it to the minimal subgraph as a necessary node that has an impact on file defects. The nodes in the minimal subgraph can be regarded as critical statements that may have defects. During the experiments, we found that the subgraph generated using GNNExplainer still contained many nodes; however, there were very few defective lines in the file. In this paper, we further measure the importance of nodes in the minimal subgraph by using centrality in SNA: the more important a line of code is, the more it contributes to the program semantics (i.e., functionality) of the implemented function(Wu et al., 2022b), so LineFlowDP uses the node importance as the risk score for the node. Specifically, we use the following three centralities to evaluate the importance of nodes in the minimal subgraph.

Degree centrality(Freeman, 1978) is the percentage of nodes connected to the current node. Degree centrality is the most direct measure of node centrality in SNA; the greater the degree of a node is, the higher the degree centrality and the more important the node is in the network. The degree centrality values are normalized by dividing by the maximum value of the node degree in the graph. The purpose of normalizing the centrality values is to standardize the ranges of the three centrality values to [0,1], which prevents one centrality value from being too large and weakens the effect of the other two centrality values. Let G denote the entire PDG and i denote one of its nodes. The degree centrality is calculated as in Equation (5), where $\text{degree}()$ denotes the degree of a node in the graph.

$$\text{degree_norm}_i = \frac{\text{degree}(i)}{\max(\text{degree}(G))} \quad (5)$$

Katz centrality(Katz, 1953) calculates the relative impact of nodes by counting the number of direct neighbours and the number of all other nodes connected through these direct neighbours.

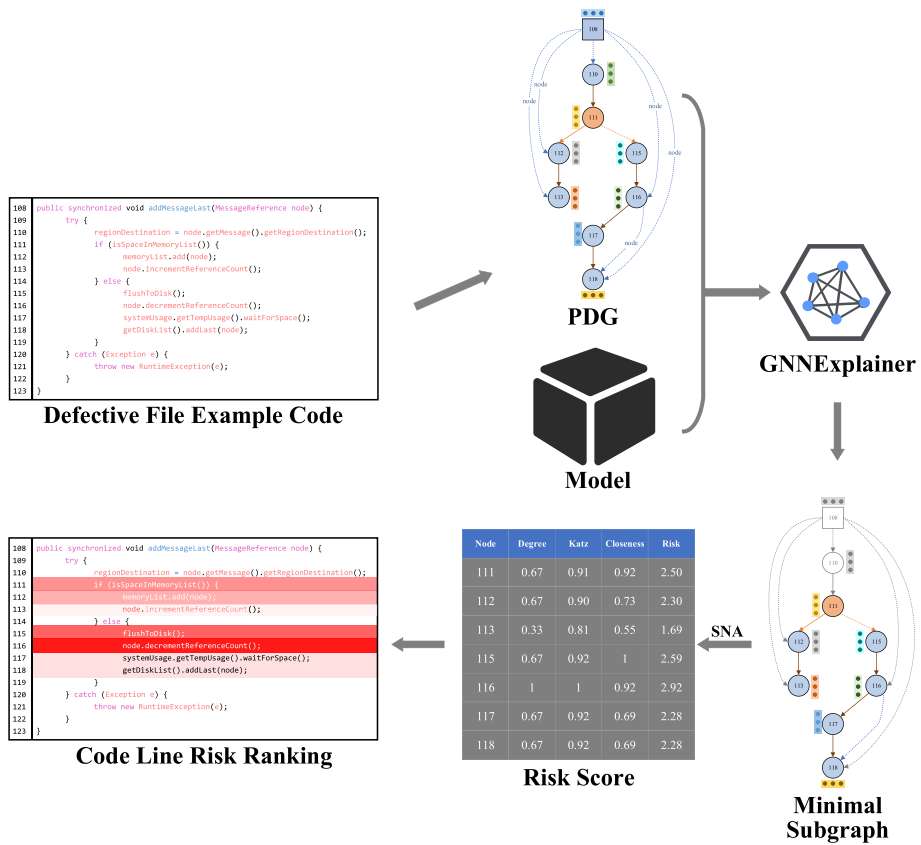


Fig. 6 Risk ranking of code lines in the defective file

$$Katz_i = \alpha \sum_j A_{ij} x_j + \beta \quad (6)$$

where A is the adjacency matrix of a graph G with eigenvalue λ . The parameter β controls the initial centrality and satisfies:

$$\alpha < \frac{1}{\max(\lambda)} \quad (7)$$

Similarly, the Katz centrality values are normalized:

$$Katz_norm_i = \frac{Katz_i}{\max(Katz(G))} \quad (8)$$

Closeness centrality(Freeman, 1978): Closeness centrality reflects the proximity of a node to all other nodes in the network. It is defined as the average of the shortest path lengths from the node to other nodes. The smaller the average shortest distance of a node is, the greater the closeness centrality of the node.

$$closeness_i = \frac{N - 1}{\sum_{i \neq j} d(i, j)} \quad (9)$$

where $d(i, j)$ is the distance between nodes i and j , N is the number of nodes in graph G . The same normalization operation is performed on the closeness centrality values:

$$closeness_norm_i = \frac{closeness_i}{\max(closeness(G))} \quad (10)$$

As shown in Equation (11), the sum of the normalized degree centrality, Katz centrality and closeness centrality values of a node in the minimal subgraph generated by GNNExplainer is defined as the importance of the node, and the importance is used as the risk score for the line of code. The higher the importance of a node, the higher the risk score of the corresponding line of code. To more accurately reflect the importance of a node, we consider three centrality values for the node. However, if one of the centrality values is small relative to the others, the contribution of this smaller value is diluted when calculating the total centrality value. To solve this problem, we normalize each centrality value and then sum it. In this way, each centrality value can have a fair impact on the total centrality value, thus reducing to some extent the data bias caused by centrality values that are too large or too small.

$$risk_score_i = degree_norm_i + Katz_norm_i + closeness_norm_i \quad (11)$$

4 Experimental Design

Our experiments were conducted on a computer with a GPU (NVIDIA GTX 3060), CPU (i7-11700K, 16 cores), Windows 10 operating system, Python 3.8 programming language, and RGCN implemented by PyTorch. We use the word embedding vector size of 100 to train Doc2Vec model. The RGCN model is trained at 1000 epochs. We used binary cross-entropy as the loss function, and the Adam optimizer (Kingma & Adam, 2014) was used to minimize the loss function.

To measure the effectiveness of LineFlowDP, we propose four research questions in this paper.

4.1 Research Questions

- **RQ1:** How accurate is LineFlowDP for file-level defect prediction?
- **Motivation:** It is worthwhile to investigate whether LineFlowDP outperforms the state-of-the-art file-level SDP methods. We compare LineFlowDP with the nine state-of-the-art file-level approaches.
- **RQ2:** In the research question, we explored the predictive performance of LineFlowDP at the line-level as follows. RQ2.1: What is the cost-effectiveness of LineFlowDP for line-level defect prediction? RQ2.2: How accurate is LineFlowDP for line-level defect prediction?
- **Motivation:** We study how well LineFlowDP performs line-level SDP. We compare LineFlowDP with the four state-of-the-art baselines.

- **RQ3:** RQ3.1: Can flow information extension benefit line-level defect prediction? RQ3.2: Can the three centrality combination approach benefit line-level defect prediction?
- **Motivation:** We designed two ablation experiments as above to explore the effectiveness of flow information extension and risk ranking methods in LineFlowDP.
- **RQ4:** How time efficient is LineFlowDP?
- **Motivation:** Efficiency is an important aspect when assessing the performance of SDP methods. This research question aims to investigate whether the LineFlowDP approach can achieve a better balance between accuracy and efficiency.

4.2 Datasets

The dataset used in this paper is the the benchmark line-level defect prediction dataset collected by Wattanakriengkrai et al.(Wattanakriengkrai et al., 2020), which consists of 32 versions of 9 open-source software systems. Table 1 presents brief statistical information about the dataset. Each release contains 731 to 8,846 files, 74,349-567,804 lines of code.

The reason we chose this dataset is because it has been widely used in previous studies on line-level software defect prediction(Wattanakriengkrai et al., 2020; Pornprasit & Tantithamthavorn, 2022). This indicates that the dataset has been validated and recognized, making it a reliable benchmark for evaluating the performance of our LineFlowDP.

4.3 Evaluation Metrics

In this paper, we use the widely used traditional metrics of area under the receiver operating characteristic (ROC) curve (AUC)(Pornprasit & Tantithamthavorn, 2022; Wang et al., 2010; Miletic et al., 2018), balanced accuracy (BA)(Pornprasit & Tantithamthavorn, 2022; Sohn et al., 2021; Khakhar & Dubey, 2022), Matthew's correlation coefficient (MCC)(Wattanakriengkrai et al., 2020; Pornprasit & Tantithamthavorn, 2022) and the geometric mean(GM) between TPR and TNR(Wang et al., 2010; Miletic et al., 2018) to evaluate the prediction performance of LineFlowDP at the file level. Recall@Top20%LOC(Wattanakriengkrai et al., 2020; Pornprasit & Tantithamthavorn, 2022), Effort@Top20%Recall(Pornprasit & Tantithamthavorn, 2022) and Initial False Alarms (IFA)(Wattanakriengkrai et al., 2020; Pornprasit & Tantithamthavorn, 2022) were used to measure the cost-effectiveness of line-level defect prediction. In practice, inspecting all

Table 1 Dataset summary information statistics

Project	#File	#LOC	#Defective File	#Defective LOC	Versions
ActiveMQ	1884-3420	142k-299k	2%-7%	0.08%-0.44%	5.0.0, 5.1.0, 5.2.0, 5.3.0, 5.8.0
Camel	1515-8846	75k-485k	2%-8%	0.09%-0.24%	1.4.0, 2.9.0, 2.10.0, 2.11.0
Derby	1963-2705	412k-533k	6%-28%	0.10%-0.63%	10.2.1.6, 10.3.1.4, 10.5.1.1
Groovy	757-884	74k-94k	2%-4%	0.10%-0.17%	1.5.7, 1.6.0.Beta_1, 1.6.0.Beta_2
HBase	1059-1834	246k-537k	7%-11%	0.17%-1.02%	0.94.0, 0.95.0, 0.95.2
Hive	1416-2662	290k-567k	6%-19%	0.31%-2.90%	0.9.0, 0.10.0, 0.12.0
JRuby	731-1614	106k-240k	2%-13%	0.03%-0.09%	1.1, 1.4, 1.5, 1.7
Lucene	805-2806	101k-342k	2%-8%	0.07%-0.39%	2.3.0, 2.9.0, 3.0.0, 3.1.0
Wicket	1672-2578	106k-165k	2%-16%	0.05%-0.46%	1.3.0.beta1, 1.3.0.beta2, 1.5.3

changed LOC in all files is probably unrealistic. Prior studies assumed that only a small proportion of the changed LOC could be inspected given the limited resources and time in practice (Ray et al., 2016; Rahman et al., 2012). Therefore, we also evaluated our approach considering a limited inspection budget (i.e., inspecting the most Top-k buggy lines of a version). We use the Top-k Accuracy (Yan et al., 2020; Zhang et al., 2022) metric to measure the accuracy of our approach in locating defective lines of code.

The AUC ranges from [0,1], where a value of 1 indicates the best model prediction, and a value of 0.5 indicates a random guess. However, it should be noted that in practical software development and testing scenarios, the AUC metric is truly applicable in real-world situations only after an exhaustive exploration of all possible thresholds (Morasca & Lavazza, 2020).

BA measures the average of the true positive rate (i.e., how many predicted defective files are correct) and the true negative rate (i.e., how many predicted nondefect files are correct). The BA is calculated as in Equation (12), where TP, TN, FP and FN are the number of true positives, true negatives, false positives and false negatives, respectively. A high BA indicates that the method can accurately predict both defective and non-defective files.

$$BA = \frac{\frac{TP}{TP+FN} + \frac{TN}{TN+FP}}{2} \quad (12)$$

The MCC is calculated as in Equation (13) to measure the correlation coefficient between the actual and predicted results. The MCC values range from [-1, 1], where an MCC value of 1 indicates the best prediction, and a value of -1 indicates that the predicted results are completely inconsistent with the actual results.

$$MCC = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (13)$$

The geometric mean (GM) between true positive rate (TPR) and true negative rate (TNR) is a comprehensive metric that calculates the geometric mean of TPR and TNR. The GM metric combines TPR and TNR by taking their geometric mean, providing an overall assessment of the model's performance. The GM value ranges between 0 and 1, where a value closer to 1 indicates a more balanced ability of the model to recognize both positive and negative instances, implying better performance.

$$GM = \sqrt{TPR * TNR} \quad (14)$$

Recall@Top20%LOC measures how many defective lines of code can be accurately found when examining the top 20% LOC of the entire version. A higher value for Recall@Top20%LOC indicates that the method ranks many actual defective lines towards the top. With a fixed amount of work, we can find more lines of code that actually have defects. A lower LOC value for Recall@Top20% indicates more clean lines of code in the top 20% of lines of code, in which case we need to devote more effort to locating defective lines of code.

Effort@Top20%Recall measures the amount of work required to find the top 20% of actual defective lines in the entire version (e.g., lines of code). The higher the value of Effort@Top20%Recall, the more effort the developer needs to spend to find the top 20% of actual defective lines in the entire version.

IFA measures the number of clean lines that developers need to check until the first defective line in each file is found (Huang et al., 2017). The lower the IFA value, the fewer

clean lines are ranked, while the higher the IFA value, the more unnecessary effort developers will spend on clean lines of code. The intuition of this metric is that if developers cannot obtain promising results (i.e., find defective lines) within the first few lines of code checked, they may stop checking further lines of code (Parnin & Orso, 2011).

Top-k accuracy is the percentage of buggy lines for which at least one actually buggy line is ranked within the Top-k position in our returned list of sorted code lines. In terms of one buggy line i , if at least one of the Top-k most likely buggy lines returned by our approach is actually the buggy location, we consider the localization to be successful, and set the top-k value of this line $Top_k(i)$ to 1; otherwise, we consider the localization as unsuccessful, and set the Top-k value $Top_k(i)$ to 0. Given a set of N buggy lines in a version V , its Top-k Accuracy is computed as $Top_k(V) = \frac{1}{N} \sum_{i=1}^N Top_k(i)$. In our experimental design, we set $k=1, k=5, k=10$ and $k=20$.

5 Analysis of Results

5.1 RQ1: How Accurate is LineFlowDP for File-level Defect Prediction?

Pornprasit et al. (Pornprasit & Tantithamthavorn, 2022) first predicted defects at the file level and further extracted the risk of defects in lines of code in the defective files. Therefore, we also predict the defects at the file level first and then extract the high-risk lines of code if the file is defective. To measure the accuracy of LineFlowDP at the file level, nine state-of-the-art file-level defect prediction methods (Li et al., 2021; Pornprasit & Tantithamthavorn, 2022; Hata et al., 2010; Wang et al., 2018; Jian et al., 2017; Dam et al., 2018; Wang et al., 2021; Pradel & Sen, 2018; Hin et al., 2022) are selected in this paper to verify the prediction performance of LineFlowDP in four traditional metrics. Similar to the experiment of Pornprasit et al. (Pornprasit & Tantithamthavorn, 2022), the first version of each project was used as the training set, the second version was used as the validation set, and the other versions were used as the test set. Finally, 14 versions were used as the test set to compare our method with these defect prediction methods. Since the authors of the original papers (Hata et al., 2010; Wang et al., 2018; Jian et al., 2017; Dam et al., 2018) did not define names for their methods, these methods are indicated by the models they used in their methods.

- (1) IVDetect (Li et al., 2021) is designed as an interpretable vulnerability detector that combines artificial intelligence for detecting vulnerabilities and an intelligent assistant for providing interpretations of vulnerabilities based on statements.
- (2) DeepLineDP (Pornprasit & Tantithamthavorn, 2022) is a deep learning-based defect prediction method that learns the neighbouring tokens and structural associations of each token to predict defective files.
- (3) Bag-of-Words (BoW) predicts defective files by using the frequency of code tokens as a text feature (Hata et al., 2010). Referring to the work of Hata et al. (Hata et al., 2010), BoW features were trained using the logistic regression classification technique, and class imbalance in the training dataset was handled by the SMOTE technique (Chawla et al., 2002).
- (4) The deep belief network (DBN) is a model that automatically learns semantic features to predict defective files (Wang et al., 2018).

- (5) A convolutional neural network (CNN) is a model that can automatically learn semantic and structural features for predicting defective files(Jian et al., 2017).
- (6) Bidirectional long short-term memory (Bi-LSTM) is a model that automatically learns semantic and syntactic features to predict defective files(Dam et al., 2018).
- (7) CodeT5(Wang et al., 2021) is an automated code generation model based on the T5 framework. It leverages large-scale pre-training and transfer learning to generate high-quality code snippets from natural language descriptions or prompts. CodeT5 has demonstrated impressive performance across various programming languages and tasks, including code summarization, translation, completion, and bug detection.
- (8) DeepBugs(Pradel & Sen, 2018) is a learning approach to name-based bug detection, which reasons about names based on a semantic representation and which automatically learns bug detectors instead of manually writing them.
- (9) LineVD(Hin et al., 2022) is a novel deep learning framework that specializes in statement-level vulnerability detection. LineVD treats statement-level vulnerability detection as a node classification task and uses GNN to capture control and data dependencies between statements. When LineVD predicts a line with a defect, we treat the corresponding file as defective.

We compared four metrics (i.e., AUC, BA, MCC and GM) that are widely used in the defect prediction field. Using $(P_{LineFlowDP} - P_{baselines})/P_{baselines} \times 100\%$ to compute the percent improvement of LineFlowDP. Then we apply the ScottKnott ESD test to cluster the distributions into statistically distinct ranks with non-negligible effect size differences(Tantithamthavorn et al., 2018a; Tantithamthavorn et al., 2016a). We employ the ScottKnott ESD test(Tantithamthavorn et al., 2016b) (using a 95% significance level) to compare the LineFlowDP method with the comparison methods. The ScottKnott test is a hierarchical clustering algorithm that ranks the distributions of values. Specifically, it assigns metrics with statistically non-significant differences in their distributions to the same rank. The ScottKnott ESD test extends this approach by considering not only statistical significance but also Cohen's d effect size (Cohen, 2013). It assigns distributions with either non-significant differences or negligible effect sizes to the same rank. The ScottKnott ESD test produces the ranking of the techniques while ensuring that (1) the magnitude of the difference for all of the distributions in each rank is negligible; and (2) the magnitude of the difference of distributions between ranks is non-negligible. The results are shown in Fig. 7. “↑” indicates that the higher the values are, the better the approach, and “↓” indicates that the lower the values are, the better the approach.

Result LineFlowDP is 14%-56% and 28%-161% more accurate than the state-of-the-art methods in terms of the median BA and the median GM, respectively. Meanwhile, LineFlowDP tied for first rank in the AUC metric.

The detailed experimental results can be found in Tables 2, 3, 4 and 5 of the Appendix. According to the results, the median AUC is 0.800 for LineFlowDP, 0.726 for CNN, 0.577 for DBN, 0.603 for BoW, 0.518 for Bi-LSTM, 0.605 for CodeT5, 0.634 for DeepBugs, 0.644 for IVDetect, 0.668 for LineVD and 0.816 for DeepLineDP. The AUC metrics of LineFlowDP outperformed the majority of state-of-the-art methods. The median BA is 0.657 for LineFlowDP, 0.532 for CNN, 0.420 for DBN, 0.428 for BoW, 0.479 for Bi-LSTM, 0.468 for CodeT5, 0.532 for DeepBugs, 0.569 for IVDetect, 0.524 for LineVD and 0.577 for DeepLineDP. The median of GM is 0.651 for our LineFlowDP, 0.367 for CNN, 0.367 for DBN, 0.349 for BoW, 0.249 for Bi-LSTM, 0.481 for CodeT5, 0.432 for DeepBugs,

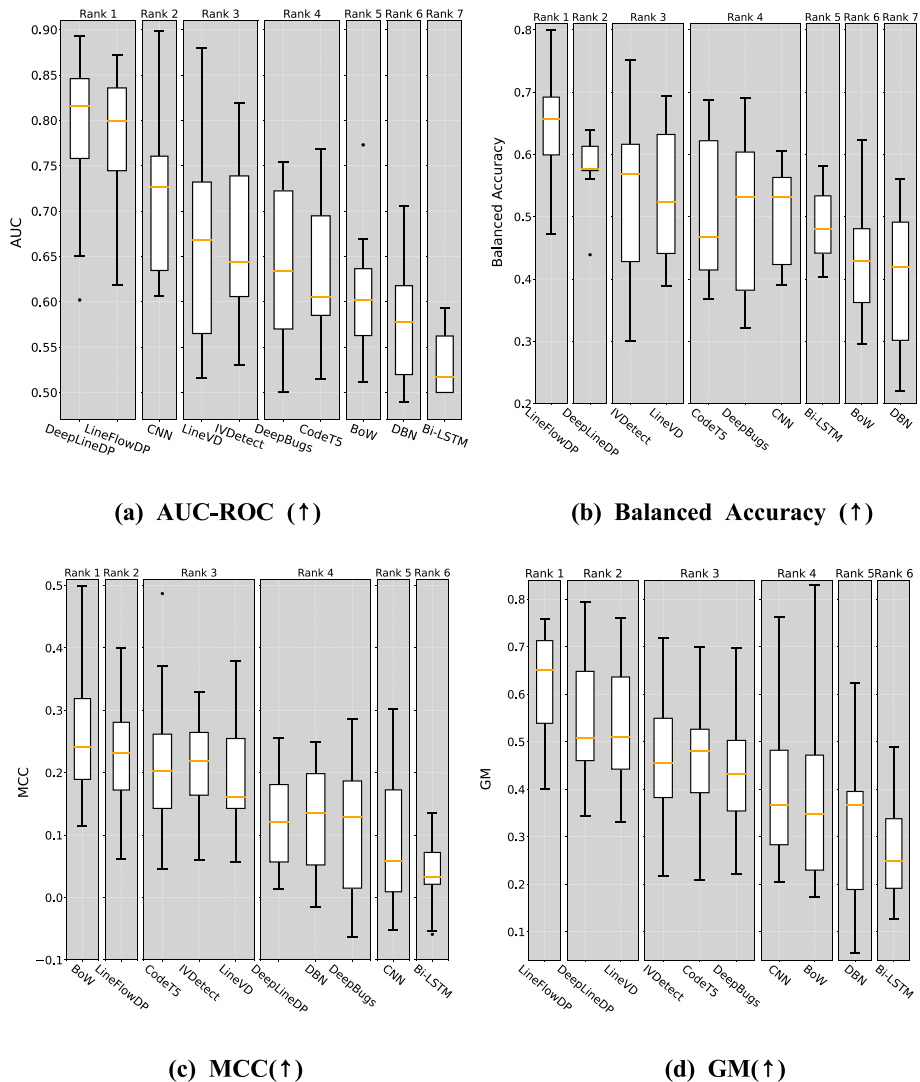


Fig. 7 ScottKnott ESD tests for AUC, BA, MCC and GM

0.456 for IVDetect, 0.510 for LineVD and 0.507 for DeepLineDP. These results show our LineFlowDP approach outperforms the state-of-the-art file-level SDP approaches. Although LineFlowDP is slightly lower on the AUC metric than DeepLineDP, the ScottKnott ESD test shows that LineFlowDP and DeepLineDP both ranked first and the performance difference is not statistically significant with non-negligible effect size. This result shows that both LineFlowDP and DeepLineDP achieved excellent performance on the AUC metric. For BA and GM metrics, the ScottKnott ESD test confirms that LineFlowDP appears at the top rank, indicating that the performance difference is statically significant with non-negligible. In addition, we observed that the median of LineFlowDP is slightly lower than that of the BoW method. Since MCC reflects the false positive rate and false negative rate of the model, our LineFlowDP method predicted more clean files as defective than the BoW method. At the

same time, we note that all the methods have lower MCC values (the highest median does not exceed 0.25). However, it is worth noting that MCC is not an entirely unbiased metric, usually presenting low values for class-imbalanced datasets(Zhu, 2020).

Answer for RQ1: *Our LineFlowDP approach excels in terms of the AUC, BA and GM metric and ranked first or tied for first rank in the ScottKnott ESD test. It performs best in the BA metric, indicating that LineFlowDP predicts file-level defects better in the presence of imbalanced data. In addition, the best GM indicates that the LineFlowDP can balance the accuracy of positive and negative samples in defect prediction, and has more comprehensive and reliable performance. Although LineFlowDP achieves lower results on the MCC metric, it still achieves respectable results relative to the vast majority of baseline methods.. These experimental results demonstrate that LineFlowDP can accurately predict defects in files.*

5.2 RQ2: What is the Performance of LineFlowDP at the Line-level?

After predicting a file as defective, LineFlowDP proceeds risk-ranks the lines of code in the defective file. Therefore, in this section, we evaluate the effectiveness of LineFlowDP's performance at the line level.

To measure the cost-effectiveness and accuracy of LineFlowDP in predicting defective lines of code, four state-of-the-art line-level defect prediction methods are selected.

- (1) N-gram: This method analyses the unnaturalness of code tokens based on the cross-entropy of each token. Previous studies have shown that N-grams are effective in identifying defective lines(Wang et al., 2016). We use the implementation of Hellendoorn and Devanbu(Hellendoorn & Devanbu, 2017) to build a cache-based language model, i.e., an enhanced N-gram model for source code detection. The cache-based language model is built by learning from nondefect code files. Similar to the procedure of Hellendoorn et al.(Hellendoorn & Devanbu, 2017), after calculating the cross-entropy of all code tokens, we calculated the average value of the cross-entropy of each line of code. Then, the risk score of each line of code token is calculated based on the average entropy value of that line of code.
- (2) ErrorProne(Aftandilian et al., 2012) is Google's static analysis tool, and it is built on the primary Java compiler (javac) according to a set of rules to check for defects in the source code. We use ErrorProne rather than other static analysis tools (e.g., FindBugs) because ErrorProne is able to detect more critical software defects (e.g., compilation defects) than FindBugs(Pornprasit & Tantithamthavorn, 2022; Aftandilian et al., 2012). If the defect is generated by ErrorProne, the line of code is considered defective. This form of detection simulates the top-down reading method, that is, developers follow the file from the top to the bottom to check the source code.
- (3) LineVD(Hin et al., 2022) is a novel deep learning framework that specializes in statement-level vulnerability detection. LineVD treats statement-level vulnerability detection as a node classification task and uses GNN to capture control and data dependencies between statements. The authors' experimental results show that LineVD has excellent vulnerability detection performance.
- (4) DeepLineDP(Pornprasit & Tantithamthavorn, 2022) is a deep learning-based method for line-level defect prediction that first generates a vector representation for the code token, then uses bidirectional GRU to learn the neighbouring tokens of the code line, and further uses hierarchical attention networks to learn the hierarchy of the source

code (i.e., the tokens constitute a line of code, and lines of code constitute a file) to predict whether the file has defects. Finally, the risk score of the code token in the defective file is calculated using the attention mechanism (Vaswani et al., 2017), and the sum of the risk scores of the code token in the code line is used as the risk score of the code line.

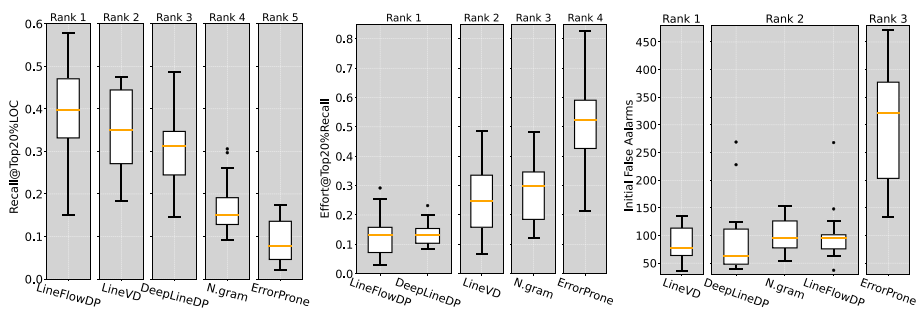
5.2.1 RQ2.1 What is the Cost-effectiveness of LineFlowDP for Line-level Defect Prediction?

Result LineFlowDP is 13%-404% and 2%-75% more cost-effective than the state-of-the-art line-level defect prediction approaches in terms of Recall@Top20%LOC and Effort@Top20%Recall.

The detailed experimental results can be found in Tables 6, 7 and 8 of the Appendix. According to the result shown in Fig. 8, the median of Recall@Top20%LOC is 0.398 for LineFlowDP, 0.151 for N.gram, 0.079 for ErrorProne, 0.351 for LineVD, and 0.314 for DeepLineDP. This finding indicates that, given a fixed amount of 20% LOC inspection effort, LineFlowDP can find the most defective lines (39.8%).

In addition, LineFlowDP achieves a median Effort@Top20%Recall of 0.131, while N.gram, ErrorProne, LineVD, and DeepLineDP achieve a median Effort@Top20%Recall of 0.299, 0.525, 0.247 and 0.133, respectively. This finding indicates that to find 20% of the actual defective lines, the LineFlowDP approach would require developers to inspect approximately 13.1% of the LOC of the entire release, while N.gram would require approximately 29.9%, ErrorProne would require approximately 52.5%, LineVD would require approximately 24.7%, and DeepLineDP would require approximately 13.3%. This result shows that it takes the least amount of effort for LineFlowDP to find the actual 20% defective lines.

We note that when checking the ranking of risk lines in each defective file, LineFlowDP achieves a median Initial False Alarms of 96, LineFlowDP's performance is inferior when compared to LineVD, N.gram and DeepLineDP, demonstrating that it may rank some non-defective lines before the first truly defective line. However, in real development scenarios where developers aim to identify as many defective lines as possible in the entire version. LineFlowDP's outstanding results on the Recall@Top20%LOC and Effort@Top20%Recall metrics demonstrate its cost-effectiveness in comprehensive code inspections.



(a) Recall@Top20%LOC(↑) (b) Effort@Top20%Recall(↓) (c) IFA(↓)

Fig. 8 ScottKnott ESD tests for three effort-aware metrics

The ScottKnott ESD test also confirms that our LineFlowDP approach always appears at the top rank in terms of Recall@Top20%LOC, Effort@Top20%Recall with non-negligible effect size. Although it achieves in IFA metric not as good as LineVD, N.gram and DeepLineDP, the reason for the low IFA metric may be caused by the use of flow information extension. This result is still acceptable relative to the number of lines of code in the entire version.

5.2.2 RQ2.2 How Accurate is LineFlowDP for Line-level Defect Prediction?

We evaluate LineFlowDP's line-level prediction accuracy using Top-k accuracy metrics, and the results are shown in Fig. 9.

Result LineFlowDP is 16%-113%, 10%-98%, 17%-139% and 8%-118% more accurate than the state-of-the-art line-level defect prediction approaches in terms of the median Top-1, Top-5, Top-10 and Top-20 accuracy.

The detailed experimental results can be found in Tables 9, 10, 11 and 12 of the Appendix. Based on the experimental results shown in Fig. 9, LineFlowDP consistently outperforms all four baselines in terms of Top-1, Top-5, Top-10 and Top-20. LineFlowDP achieves promising localization results, e.g., Top-1 is 0.198 meaning that LineFlowDP can locate the defective lines at the first place in 19.8% of all defective lines. The median values of the evaluation metrics for LineFlowDP are as follows: Top-1 accuracy is 0.198, Top-5 accuracy is 0.376, Top-10 accuracy is 0.593 and Top-20 accuracy is 0.675.

Comparing LineFlowDP to the N.gram baseline, LineFlowDP achieves higher median values for Top-1 accuracy (0.130 vs. 0.198), Top-5 accuracy (0.229 vs. 0.376), Top-10 accuracy (0.361 vs. 0.593), and Top-20 accuracy (0.438 vs. 0.675). This demonstrates the superior performance of LineFlowDP over the N.gram method.

Similarly, in comparison to the ErrorProne baseline, LineFlowDP exhibits higher median values for Top-1 accuracy (0.093 vs. 0.198), Top-5 accuracy (0.190 vs. 0.376), Top-10 accuracy (0.248 vs. 0.593), and Top-20 accuracy (0.310 vs. 0.675).

In comparison to the LineVD baseline, LineFlowDP demonstrates better performance with higher median values for Top-1 accuracy (0.160 vs. 0.198), Top-5 accuracy (0.343 vs. 0.376), Top-10 accuracy (0.506 vs. 0.593), and Top-20 accuracy (0.625 vs. 0.675).

Finally, when compared to the DeepLineDP baseline, LineFlowDP still outperforms it in terms of Top-1 accuracy (0.170 vs. 0.198), Top-5 accuracy (0.282 vs. 0.376), Top-10 accuracy (0.435 vs. 0.593), and Top-20 accuracy (0.558 vs. 0.675).

Meanwhile, the ScottKnott ESD tests also confirms that LineFlowDP always appears at the top rank in terms of the four metrics with non-negligible effect size.

Overall, the results indicate that LineFlowDP consistently achieves better performance compared to the four state-of-the-art line-level prediction methods in terms of Top-k accuracy. This finding underscores the superiority and effectiveness of LineFlowDP in line-level defect prediction.

Answer for RQ2: *Our LineFlowDP approach achieves the highest median scores for the Recall@Top20%LOC, Effort@Top20%Recall, Top-k accuracy metrics and ranks first in the ScottKnott ESD tests. This indicates that LineFlowDP is able to find more defective lines of code and requires the least amount of effort to find the actual top 20% of defective lines of code in the entire version. Additionally, LineFlowDP offers higher accuracy in the entire version. Although LineFlowDP performs less well than the other three methods on the IFA metric, it excels on the Recall@Top20%LOC, Effort@Top20%Recall and Top-k accuracy metrics also indicates that LineFlowDP remains cost-effective and better accuracy.*

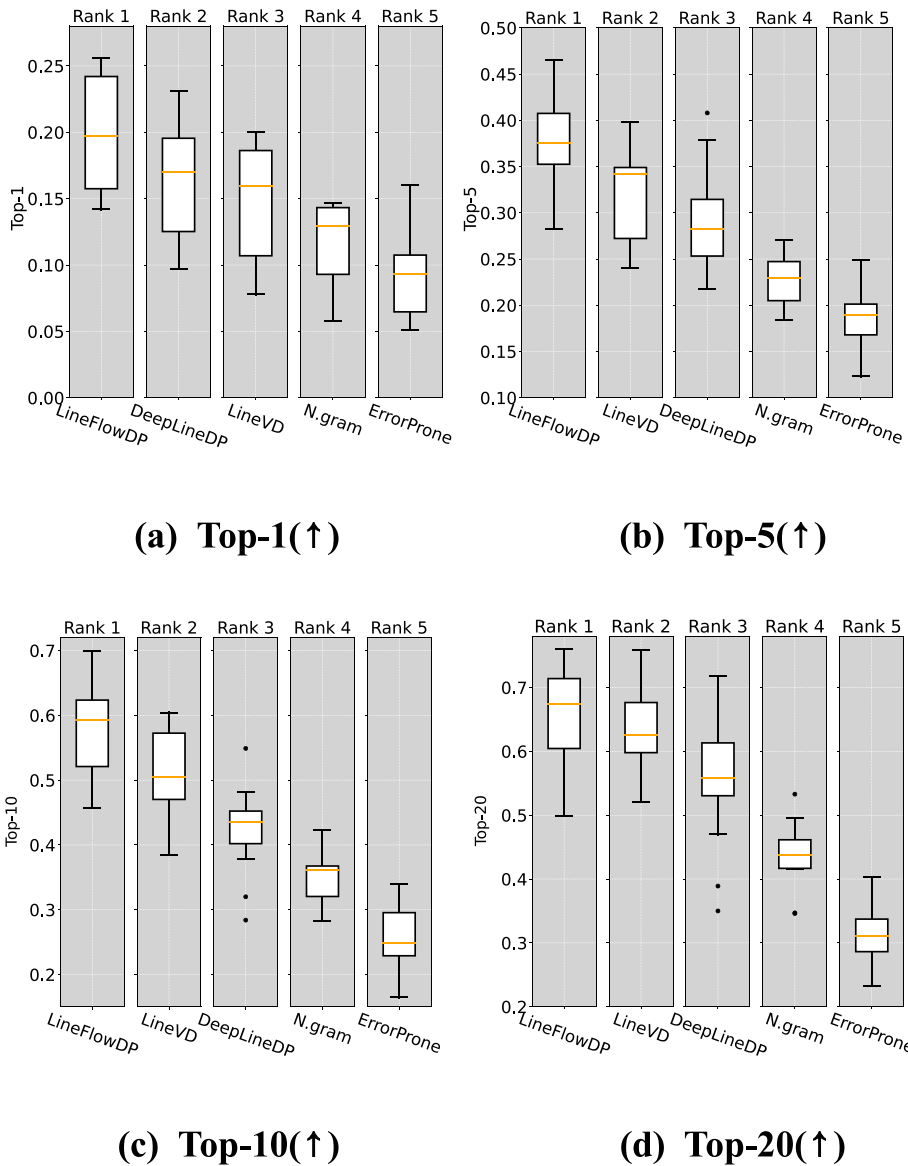


Fig. 9 ScottKnott ESD tests for Top-k accuracy

5.3 RQ3: Ablation Experiments

5.3.1 RQ3.1: Can Flow Information Extension Benefit Line-level Defect Prediction?

Since our final prediction granularity is at the line level, we compare the effectiveness of the flow information extension method at the line level. In this section, we compare

the prediction results by designing ablation experiments. Specifically, we devised the following three ways to explore how extracting semantic information is most effective.

Base: Extraction of semantic information based on the current line of code without using any extended information.

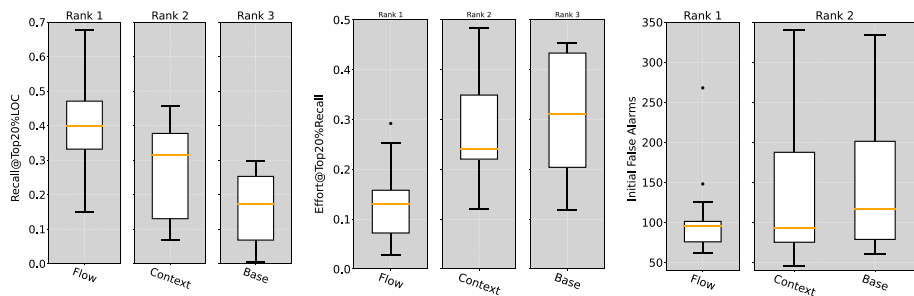
Context: Contextual neighbouring extension, splicing the first two lines of code, the current line of code and the last two lines of code for simple semantic extension.

Flow: Semantic extension of code lines by using flow information in the PDG, i.e., the approach for semantic extension of lines of code we proposed in Section 3.1.

The detailed experimental results can be found in Tables 13, 14 and 15 of the Appendix. As the result in Fig. 10, in the median, the Flow strategy achieves 0.398 and 0.131 for Recall@Top20%LOC and Effort@Top20%Recall, respectively, which are better values than those of the Base and Context strategies. Flow's median on the IFA metric is lower than Context, but this difference is small (only 3 LOC difference). At the same time, we observe that the ScottKnott ESD tests also confirm that the Flow strategy appears at the top rank in terms of three effort-aware metrics with non-negligible effect size. Thus, the semantic extension of code lines by flow information in PDG is effective and outperforms the without information extension and contextual neighbouring information extension strategies.

5.3.2 RQ3.2: Can the Three Centrality Combination Approach Benefit Line-level Defect Prediction?

To verify the superiority of the proposed method for ranking the risk of code lines using the graph interpreter GNNExplainer and SNA, we conducted ablation experiments on the three centrality values used in the SNA method, splitting the combined three centrality values into individual values and verifying the contribution of each centrality value to the risk ranking in the experimental results (D for degree centrality, K for Katz centrality, C for closeness centrality, and DKC for the combination of these three centrality values used in the paper). Since the method in this paper does not have line-level granularity without using SNA to calculate node importance, at least one centrality value needs to be preserved for this part of the ablation experiment. The results of the three centrality ablation experiments are shown in Fig. 11.



(a) Recall@Top20%LOC(↑)

(b) Effort@Top20%Recall (↓)

(c) IFA(↓)

Fig. 10 Comparison of three effort-aware metrics for RQ3.1

The detailed experimental results can be found in Tables 16, 17 and 18 of the Appendix. The results show that the use of a single centrality value on some versions achieves slightly better results than the combined centrality. However, each centrality works differently for different projects and evaluation metrics. For example, the Katz centrality value is better overall for Effort@Top20%Recall, and closeness centrality works well on the IFA metrics on the Camel project. Overall, the combination of the three centralities is much better overall than using a single centrality. In addition, choosing different centrality values for each version and each evaluation metric individually requires a significant amount of time and effort, making the use of a combination of centrality values more cost-effective than the use of a single centrality value. Meanwhile, the ScottKnott ESD tests show that the three centrality combination approach ranked highest in all three metrics, and achieved the best results.

Answer for RQ3: *The flow information extension and three centrality combination both had a beneficial impact on LineFlowDP, and they all ranked first or tied for first in ScottKnott ESD tests.*

5.4 RQ4: How Time Efficient is LineFlowDP?

The median time efficiency results are shown in Fig. 12. Our LineFlowDP and IVDetect need to train two models (one for the prediction model and the other for the GNNExplainer), and we add these two times as the total training time. For the CodeT5 model, we downloaded the pre-trained base model and fine-tuned it, and then applied it to defect prediction.

The detailed experimental results can be found in Table 19 of the Appendix. From Fig. 12, it can be observed that the training and test time of the LineFlowDP model are both reasonable. Specifically, the median training time for a model is approximately 591.3 seconds, while predicting labels in the test version using the model takes approximately 13.3 seconds. Although the training time of our LineFlowDP model is longer than that of most models, the difference in prediction time is not significant. In fact, because deep learning models can be trained offline, their training cost may not be that important (Cao et al., 2022). Moreover, LineFlowDP demonstrates satisfactory prediction performance, which leads us to believe that LineFlowDP achieves a good balance between accuracy and time efficiency.

Answer for RQ4: *Despite the long training time needed, LineFlowDP has a relatively short prediction time and achieves good predictions. LineFlowDP makes a trade-off between accuracy and efficiency.*

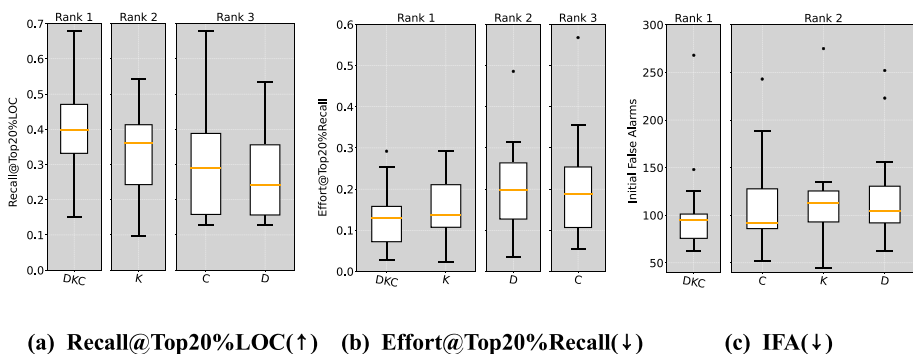


Fig. 11 Comparison of three effort-aware metrics for RQ3.2

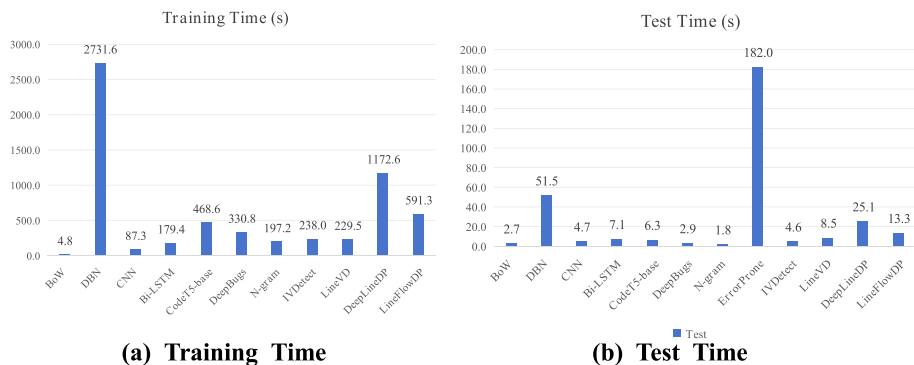


Fig. 12 The median of time efficiency (seconds)

6 Discussion

6.1 Why Does LineFlowDP Work?

First, LineFlowDP extends the semantic information of a single line of code by using the flow information in the PDG, and since the extended statements have data dependency or control dependency with the current lines of code, the extended statements are closely semantically related to the current statements. LineFlowDP extended single line code into multiple lines, with the extended statements maintaining a strong dependency on the current statement. It then uses word embedding techniques to convert these statements into vectors within a low-dimensional space. This effectively tackles the issue of a single line of code containing insufficient semantic information for semantic differentiation. Second, for IVDetect, which is based on post-hoc interpretable methods for vulnerability detection, the method extracts the minimal subgraph associated with the predicted vulnerability as a key statement by using GNNExplainer. We use SNA to calculate the importance of nodes in the smallest subgraph as the defect risk of the corresponding line of code, which effectively solves the coarse-grained problem in IVDetect and thus achieves line-level defect prediction.

In this paper, the use of SNA to calculate node importance is inspired by the study of Wu et al. (Wu et al., 2022b), who used three centralities in SNA to detect vulnerabilities by constructing images corresponding to three channels and multiplying the three centrality values with the three channels corresponding to the vector of the line of code. The effectiveness of the SNA method in vulnerability detection has been demonstrated in their research, and we apply it to defect prediction research from a novel perspective.

6.2 Advantages of Our LineFlowDP for Line-level Defect Prediction

In previous studies on line-level defect prediction (Pornprasit & Tantithamthavorn, 2022; Ray et al., 2016; Wang et al., 2016; Hellendoorn & Devanbu, 2017), researchers calculated the entropy value of lines of code by using the entropy of code tokens or the attention score

of code statements by using the attention mechanism, which are unreliable in some scenarios. They account for lines of code in the file that are not directly involved in the execution process (e.g., block terminators such as “}”, “import”, “package” and “else” statements), these statements are unlikely to introduce post-release defects because there are no control or dependency relationships between them and other statements. Static detection tools (such as ErrorProne(Aftandilian et al., 2012)) require predefined rules to determine whether a line of code meets the requirements; if not, the line of code is considered defective. However, this static tool relies too much on a priori knowledge because defects that are not in the rules cannot be detected. LineFlowDP extracts the PDG of the source code, and the statements corresponding to the nodes in the PDG are directly involved in the execution of the code. By using the flow information in the PDG to extend the semantics of code lines, the centrality values obtained by using GNNExplainer and SNA for the files predicted to be defective are extracted as the risk scores of the corresponding code lines, thus achieving line-level defect prediction. We have validated the effectiveness of LineFlowDP by designing comprehensive experiments.

As shown in Fig. 13 for a case study we conducted, we show the prediction results of LineFlowDP and the baseline methods, where the 116th line is the actual defective line, and we normalize the final risk scores in order to show the results more visually. DeepLineDP and N-gram are line-level defect prediction methods based on model score implementations, and it can be seen that they treat statements that do not have control dependencies or data dependencies with other lines as equally likely to be at risk, e.g., the N-gram method treats line 109 as a defective line, which is impractical since a try statement alone does not introduce a defect. Similarly, since the DeepLineDP method simply learns the structural relationships between tokens and learns the features of some lines that do not interact with other lines, it incorrectly treats line 108 as a defective line, while line 116 is only assigned a lower risk score. Since the code in the sample snippet does not exist in the pre-defined defect matching rules, ErrorProne treats them all as non-defective lines. Compared to the LineVD method, our LineFlowDP predicts the defective lines more accurately. This is because LineFlowDP accurately captures the frequent interactions between line 116 and other lines through a combination of GNNExplainer and social network analysis, and thus it contributes the most to the defective module.

//Line-Level Defect Prediction by LineFlowDP and Baselines						
//Sample Code Snippet from ActiveMQ	LineFlowDP	DeepLineDP	LineVD	ErrorProne	N-gram	Ground Truth
108 public synchronized void addMessageLast(MessageReference node) {	0	1	0	0	0.067	0
109 try {	0	0.113	0	0	1	0
110 regionDestination = node.getMessage().getRegionDestination();	0	0.127	0	0	0.842	0
111 if (isSpaceInMemoryList()) {	0.856	0.002	0	0	0.727	0
112 memoryList.add(node);	0.788	0.013	0.924	0	0.233	0
113 node.incrementReferenceCount();	0.579	0.064	0.919	0	0.659	0
114 } else {	0	0.037	0	0	0.766	0
115 flushToDisk();	0.887	0.014	0.471	0	0.805	0
116 node.decrementReferenceCount();	1	0.022	0.519	0	0.598	1
117 systemUsage.getTempUsage().waitForSpace();	0.781	0.011	1	0	0.010	0
118 getDiskList().addLast(node);	0.781	0.014	0.324	0	0.543	0
119 }	0	0	0	0	0.474	0
120 } catch (Exception e) {	0	0.185	0	0	0.158	0
121 throw new RuntimeException(e);	0	0.139	0	0	0.345	0
122 }	0	0	0	0	0.968	0
123 }	0	0	0	0	0.071	0

Fig. 13 A case study of our LineFlowDP and baselines

Overall, LineFlowDP enables more cost-effective line-level defect prediction with a small amount of a priori knowledge (e.g., labelled defective files and defective statements in defect files) than four state-of-the-art line-level SDP methods.

7 Threats to Validity

7.1 Threats to Internal Validity

Internal validity threats in our study primarily revolve around the influence of hyperparameter configurations on the performance of our LineFlowDP models. The efficacy of our models could be swayed by varying weights, which were manually adjusted during our experiments. Nonetheless, fine-tuning parameter settings for deep learning methodologies is a process that demands significant computational resources. We use the following hyperparameter when fine-tuning our LineFlowDP models:

First, the learning rate is the step size for updating the parameters along the direction of gradient descent at each step when we optimize the model, and we set the learning rate = $\{0.01, 0.001, 0.0001\}$. Second, the batch size is the number of samples we use in each training iteration, and we set the batch size = $\{64, 128, 256\}$. Then, dropout is the percentage of neurons we randomly turn off during training to prevent overfitting, and we set the dropout = $\{0, 0.2, 0.5\}$. In addition, the number of hidden layer neurons (hidden_dim) is the number of neurons we set in each hidden layer, and we set the number of hidden layer neurons = $\{32, 64, 128\}$. The early-stopping strategy is a method to prevent model overfitting by stopping training when the validation loss does not improve within a certain number of consecutive training cycles (we set it to 100 epochs).

7.2 Threats to External Validity

- (1) We conducted experiments only on 32 versions of 9 open-source Java projects. However, since the PDG is a universal graphical representation of programming languages, and Doc2Vec can learn representations for any code or text, the RGCN model can automatically mine defect-related features through the extraction of the above structural and semantic information. Therefore, it is easy to extend the LineFlowDP method to other projects or programming languages.
- (2) We only validate LineFlowDP on cross-version SDP. LineFlowDP may not be suitable for cross-project SDP and just-in-time SDP for the following reasons: For cross-project SDP, researchers usually use data from other projects to train the model because of insufficient training data from the target project. However, the differences in application domains, development processes, and programming languages across projects, lead to significant distributional differences between different project datasets. This variability is reflected in our approach in the structural and semantic information of the PDG. Therefore, the performance of LineFlowDP on cross-project SDP may be affected by these differences. For just-in-time SDP, the structure of the code is based on commits. However, since the commits are only partial code snippets, these commits may not constitute complete files, preventing us from extracting an entire PDG. Therefore, we need to adopt an alternative approach to characterize these commits in order to learn structural and semantic features using LineFlowDP.

7.3 Threats to Construct Validity

- (1) In implementing the baseline approach compared in this paper, the inevitable problem is that different parameters in various models will have some impact on the final results. In this paper, we followed the ideas of the approach proposed in the baseline paper and their open-source code for experimental replication to minimize this impact.
- (2) We used only four traditional metrics that are less prone to class imbalance problems (Tantithamthavorn et al., 2018b), three effort-aware evaluation metrics and Top-k accuracy that are widely used by researchers (Wan et al., 2018b). These evaluation metrics may not be able to fully evaluate the prediction performance of LineFlowDP, and more evaluation metrics will be considered in future work.
- (3) We use LOC as a measure of developer effort when evaluating line-level predictive performance. While this is a common practice, in real-world scenarios, LOC may not accurately reflect the amount of developer effort. In future work, more factors will be considered for measuring the amount of work put into the metric, such as circle complexity.

8 Conclusion

In this paper, we proposed LineFlowDP, a deep learning method that learns the structural and semantic information of source code to predict whether files are defective and further ranks the risk of each line of code in defective files. The experimental results show that LineFlowDP is 14%-56% and 28%-161% more accurate than the state-of-the-art methods in terms of the median BA and the median GM. Meanwhile, LineFlowDP tied for first rank in the ScottKnott ESD tests on the AUC metric, indicating that LineFlowDP has higher accuracy in predicting defects in files. The improvement in line-level defect prediction over the four state-of-the-art methods in Recall@Top20%LOC is 13%-404%, indicating that LineFlowDP is more cost effective in ranking the risk of code lines.

We also propose using the flow information in the PDG to extend the single line of code semantics. The lines of code extended with flow information are closely dependent on the original lines of code, so the extended semantic information is more relevant to the original code, and the ablation experiment proves that the flow information extension has better cost-effectiveness compared with the without any extended information and contextual semantic information extension. At the same time, we further verify that the SNA method can be used to determine the importance of lines of code. In the PDG, the more important lines of code have a higher risk scores with a higher possibility of defects, and the good experimental results of defect prediction at the line-level prove the effectiveness of this method.

Appendix

In this section, we present detailed experimental results for the four research questions. **Bold and shaded** values indicate the method that performs best on each dataset, while Win/Tie/Loss (W/T/L) indicates the number of datasets on which LineFlowDP outperforms/equally performs/worse than the other methods in terms of the metric values. “↑” indicates that the higher the values are, the better the approach, and “↓” indicates that the lower the values are, the better the approach.

Detailed Results for RQ1

Table 2 Comparison of AUC metric(↑)

test_version	CNN	DBN	BoW	Bi-LSTM	CodeT5	DeepBugs	IVDetect	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.726	0.688	0.592	0.524	0.761	0.693	0.819	0.880	0.893	0.872
activemq-5.3.0	0.727	0.620	0.644	0.567	0.716	0.747	0.745	0.737	0.851	0.846
activemq-5.8.0	0.717	0.611	0.635	0.548	0.631	0.638	0.629	0.797	0.881	0.804
camel-2.10.0	0.620	0.504	0.512	0.500	0.603	0.564	0.750	0.516	0.824	0.618
camel-2.11.0	0.620	0.512	0.637	0.500	0.585	0.509	0.530	0.552	0.823	0.728
derby-10.5.1.1	0.898	0.706	0.773	0.584	0.515	0.501	0.611	0.533	0.808	0.743
groovy-1.6_BETA_2	0.620	0.516	0.669	0.500	0.585	0.630	0.574	0.648	0.602	0.795
hbase-0.95.2	0.606	0.581	0.539	0.500	0.517	0.594	0.684	0.777	0.651	0.749
hive-0.12.0	0.678	0.628	0.616	0.500	0.607	0.520	0.659	0.682	0.766	0.814
jruby-1.5.0	0.787	0.559	0.556	0.579	0.767	0.754	0.572	0.544	0.856	0.842
jruby-1.7.0.preview1	0.762	0.490	0.583	0.593	0.598	0.732	0.720	0.654	0.758	0.663
lucene-3.0.0	0.734	0.579	0.590	0.501	0.768	0.588	0.614	0.707	0.742	0.817
lucene-3.1	0.756	0.531	0.544	0.516	0.619	0.663	0.768	0.717	0.758	0.767
wicket-1.5.3	0.799	0.576	0.613	0.519	0.569	0.734	0.604	0.604	0.831	0.849
W/T/L	11/0/3	14/0/0	13/0/1	14/0/0	14/0/0	13/0/1	11/0/3	12/0/2	6/0/8	-
Median	0.726	0.577	0.603	0.518	0.605	0.634	0.644	0.668	0.816	0.800

Table 3 Comparison of BA metric(↑)

test_version	CNN	DBN	BoW	Bi-LSTM	CodeT5	DeepBugs	IVDetect	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.422	0.501	0.377	0.549	0.434	0.600	0.459	0.650	0.577	0.727
activemq-5.3.0	0.545	0.477	0.522	0.449	0.455	0.691	0.587	0.654	0.639	0.799
activemq-5.8.0	0.541	0.434	0.480	0.408	0.390	0.547	0.598	0.527	0.626	0.695
camel-2.10.0	0.491	0.249	0.305	0.403	0.368	0.375	0.646	0.574	0.575	0.473
camel-2.11.0	0.427	0.284	0.448	0.581	0.472	0.336	0.419	0.389	0.574	0.586
derby-10.5.1.1	0.606	0.561	0.604	0.481	0.463	0.573	0.752	0.694	0.622	0.548
groovy-1_6_BETA_2	0.413	0.516	0.623	0.540	0.687	0.605	0.611	0.521	0.574	0.673
hbase-0.95.2	0.402	0.440	0.366	0.454	0.383	0.516	0.622	0.439	0.439	0.606
hive-0.12.0	0.390	0.496	0.481	0.578	0.688	0.321	0.415	0.461	0.628	0.667
jruby-1.5.0	0.540	0.324	0.318	0.439	0.683	0.403	0.550	0.578	0.577	0.646
jruby-1.7.0.preview1	0.523	0.221	0.361	0.512	0.552	0.321	0.301	0.439	0.563	0.633
lucene-3.0.0	0.569	0.394	0.434	0.478	0.562	0.641	0.443	0.402	0.586	0.597
lucene-3.1	0.596	0.294	0.295	0.413	0.408	0.501	0.618	0.660	0.560	0.683
wicket-1.5.3	0.583	0.406	0.423	0.514	0.642	0.642	0.423	0.447	0.583	0.697
W/T/L	12/0/2	13/0/1	13/0/1	14/0/0	11/0/3	12/0/2	11/0/3	12/0/2	12/0/2	-
Median	0.532	0.420	0.428	0.479	0.468	0.532	0.569	0.524	0.577	0.657

Table 4 Comparison of MCC metric(↑)

test_version	CNN	DBN	BoW	Bi-LSTM	CodeT5	DeepBugs	IVDetect	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.192	0.250	0.214	0.052	0.200	0.209	0.248	0.156	0.136	0.297
activemq-5.3.0	0.302	0.214	0.366	0.020	0.271	0.060	0.173	0.165	0.256	0.376
activemq-5.8.0	0.302	0.142	0.328	0.043	0.148	0.137	0.060	0.379	0.255	0.232
camel-2.10.0	-0.004	0.018	0.114	0.033	0.141	0.000	0.209	0.086	0.047	0.061
camel-2.11.0	-0.004	0.049	0.258	0.085	0.046	0.183	0.123	0.176	0.045	0.125
derby-10.5.1.1	0.298	0.221	0.394	0.135	0.487	-0.064	0.163	0.145	0.210	0.231
groovy-1_6_BETA_2	0.044	0.176	0.499	0.000	0.206	0.286	0.270	0.157	0.014	0.271
hbase-0.95.2	-0.052	0.144	0.186	-0.054	0.266	0.283	0.329	0.251	0.105	0.098
hive-0.12.0	0.114	0.206	0.269	-0.059	0.371	-0.024	0.167	0.334	0.186	0.173
jruby-1.5.0	0.017	0.090	0.180	0.079	0.061	0.122	0.074	0.142	0.158	0.172
jruby-1.7.0.preview1	0.099	-0.015	0.224	0.032	0.249	0.120	0.230	0.057	0.102	0.261
lucene-3.0.0	0.074	0.129	0.291	0.080	0.213	0.000	0.270	0.072	0.072	0.400
lucene-3.1	0.009	0.030	0.160	0.025	0.078	0.188	0.245	0.256	0.052	0.284
wicket-1.5.3	0.010	0.062	0.199	0.025	0.179	0.140	0.330	0.359	0.166	0.228
W/T/L	12/0/2	12/0/2	6/0/8	14/0/0	10/0/4	11/0/3	11/0/3	8/0/6	11/0/3	-
Median	0.059	0.136	0.241	0.033	0.203	0.130	0.220	0.161	0.120	0.232

Table 5 Comparison of GM metric(↑)

test_version	CNN	DBN	BoW	Bi-LSTM	CodeT5	DeepBugs	IVDetect	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.302	0.416	0.447	0.127	0.591	0.440	0.566	0.548	0.597	0.706
activemq-5.3.0	0.267	0.397	0.333	0.342	0.489	0.222	0.411	0.761	0.666	0.721
activemq-5.8.0	0.332	0.368	0.490	0.184	0.428	0.367	0.495	0.699	0.514	0.728
camel-2.10.0	0.224	0.160	0.232	0.252	0.368	0.466	0.567	0.449	0.425	0.465
camel-2.11.0	0.348	0.055	0.480	0.326	0.210	0.515	0.430	0.331	0.529	0.553
derby-10.5.1.1	0.762	0.623	0.829	0.246	0.469	0.647	0.466	0.438	0.497	0.651
groovy-1_6_BETA_2	0.440	0.096	0.389	0.182	0.474	0.350	0.373	0.421	0.794	0.758
hbase-0.95.2	0.566	0.276	0.229	0.366	0.528	0.424	0.445	0.678	0.491	0.411
hive-0.12.0	0.277	0.390	0.364	0.318	0.496	0.333	0.358	0.473	0.500	0.650
jruby-1.5.0	0.204	0.134	0.173	0.382	0.381	0.400	0.218	0.557	0.450	0.715
jruby-1.7.0.preview1	0.494	0.299	0.226	0.175	0.531	0.444	0.263	0.440	0.665	0.401
lucene-3.0.0	0.446	0.366	0.259	0.235	0.699	0.697	0.499	0.571	0.437	0.547
lucene-3.1	0.387	0.388	0.205	0.214	0.269	0.298	0.719	0.471	0.344	0.698
wicket-1.5.3	0.539	0.509	0.595	0.489	0.521	0.527	0.687	0.658	0.688	0.536
W/T/L	10/0/4	14/0/0	12/0/2	14/0/0	11/0/3	10/0/4	10/0/4	9/0/5	10/0/4	-
Median	0.367	0.367	0.349	0.249	0.481	0.432	0.456	0.510	0.507	0.651

Detailed Results for RQ2

Table 6 The results of Recall@Top20%LOC(↑)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.141	0.155	0.344	0.300	0.565
activemq-5.3.0	0.165	0.026	0.237	0.338	0.502
activemq-5.8.0	0.118	0.154	0.312	0.238	0.470
camel-2.10.0	0.297	0.021	0.191	0.196	0.343
camel-2.11.0	0.192	0.138	0.183	0.350	0.422
derby-10.5.1.1	0.092	0.101	0.473	0.472	0.390
groovy-1_6_BETA_2	0.144	0.055	0.290	0.324	0.407
hbase-0.95.2	0.112	0.062	0.358	0.208	0.578
hive-0.12.0	0.260	0.095	0.440	0.303	0.152
jrubby-1.5.0	0.306	0.043	0.476	0.265	0.264
jrubby-1.7.0.preview1	0.189	0.059	0.265	0.146	0.328
lucene-3.0.0	0.158	0.129	0.446	0.325	0.471
lucene-3.1	0.128	0.029	0.366	0.486	0.352
wicket-1.5.3	0.129	0.173	0.449	0.483	0.150
W/T/L	12/0/2	13/0/1	9/0/5	9/0/5	-
Median	0.151	0.079	0.351	0.314	0.398

Table 7 The results of Effort@Top20%Recall(↓)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.170	0.587	0.255	0.148	0.029
activemq-5.3.0	0.121	0.461	0.213	0.092	0.092
activemq-5.8.0	0.279	0.516	0.339	0.169	0.182
camel-2.10.0	0.245	0.254	0.485	0.200	0.164
camel-2.11.0	0.389	0.213	0.384	0.118	0.050
derby-10.5.1.1	0.174	0.600	0.147	0.083	0.131
groovy-1_6_BETA_2	0.165	0.489	0.192	0.136	0.051
hbase-0.95.2	0.333	0.534	0.112	0.156	0.140
hive-0.12.0	0.218	0.582	0.108	0.120	0.292
jruby-1.5.0	0.351	0.592	0.257	0.142	0.253
jruby-1.7.0.preview1	0.484	0.415	0.372	0.232	0.134
lucene-3.0.0	0.321	0.827	0.239	0.129	0.073
lucene-3.1	0.319	0.238	0.325	0.083	0.072
wicket-1.5.3	0.420	0.639	0.067	0.099	0.131
W/T/L	13/0/1	14/0/0	11/0/3	8/1/5	-
Median	0.299	0.525	0.247	0.133	0.131

Table 8 The results of IFA(↓)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	96	134	111	45	96
activemq-5.3.0	54	472	69	44	37
activemq-5.8.0	124	356	37	48	71
camel-2.10.0	127	289	136	64	99
camel-2.11.0	77	309	128	48	268
derby-10.5.1.1	127	181	74	124	102
groovy-1_6_BETA_2	132	180	80	121	90
hbase-0.95.2	153	171	114	83	126
hive-0.12.0	94	335	82	63	95
jruby-1.5.0	79	420	35	269	62
jruby-1.7.0.preview1	71	270	120	228	62
lucene-3.0.0	63	383	69	40	99
lucene-3.1	84	360	62	76	90
wicket-1.5.3	122	442	42	52	148
W/T/L	8/1/5	14/0/0	4/0/10	5/0/9	-
Median	95	322	77	64	96

Table 9 The results of Top-1 Accuracy(↑)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.131	0.160	0.155	0.185	0.245
activemq-5.3.0	0.096	0.112	0.164	0.205	0.256
activemq-5.8.0	0.146	0.109	0.135	0.199	0.233
camel-2.10.0	0.146	0.103	0.119	0.231	0.142
camel-2.11.0	0.107	0.064	0.078	0.109	0.153
derby-10.5.1.1	0.129	0.109	0.200	0.173	0.256
groovy-1_6_BETA_2	0.145	0.051	0.103	0.167	0.233
hbase-0.95.2	0.130	0.098	0.164	0.119	0.188
hive-0.12.0	0.086	0.102	0.188	0.211	0.151
jruby-1.5.0	0.058	0.067	0.189	0.098	0.176
jruby-1.7.0.preview1	0.138	0.076	0.085	0.149	0.155
lucene-3.0.0	0.092	0.089	0.082	0.185	0.165
lucene-3.1	0.147	0.053	0.197	0.144	0.251
wicket-1.5.3	0.131	0.160	0.155	0.185	0.207
W/T/L	13/0/1	14/0/0	12/0/2	11/0/3	-
Median	0.130	0.093	0.160	0.170	0.198

Table 10 The results of Top-5 Accuracy($\uparrow$)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.231	0.243	0.346	0.287	0.377
activemq-5.3.0	0.184	0.190	0.350	0.364	0.465
activemq-5.8.0	0.245	0.249	0.271	0.315	0.411
camel-2.10.0	0.271	0.238	0.269	0.408	0.282
camel-2.11.0	0.227	0.202	0.240	0.237	0.324
derby-10.5.1.1	0.248	0.199	0.342	0.278	0.453
groovy-1_6_BETA_2	0.231	0.129	0.278	0.305	0.397
hbase-0.95.2	0.226	0.195	0.369	0.218	0.397
hive-0.12.0	0.191	0.189	0.398	0.379	0.347
jruby-1.5.0	0.186	0.123	0.395	0.260	0.352
jruby-1.7.0.preview1	0.261	0.171	0.264	0.251	0.354
lucene-3.0.0	0.202	0.142	0.276	0.313	0.360
lucene-3.1	0.262	0.167	0.343	0.237	0.453
wicket-1.5.3	0.214	0.187	0.343	0.260	0.375
W/T/L	14/0/0	14/0/0	12/0/2	12/0/2	-
Median	0.229	0.190	0.343	0.282	0.376

Table 11 The results of Top-10 Accuracy(↑)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.368	0.340	0.467	0.378	0.520
activemq-5.3.0	0.350	0.248	0.547	0.445	0.656
activemq-5.8.0	0.402	0.331	0.490	0.450	0.615
camel-2.10.0	0.366	0.303	0.517	0.481	0.457
camel-2.11.0	0.311	0.301	0.385	0.399	0.481
derby-10.5.1.1	0.423	0.242	0.591	0.411	0.619
groovy-1_6_BETA_2	0.361	0.226	0.510	0.459	0.625
hbase-0.95.2	0.404	0.238	0.501	0.284	0.643
hive-0.12.0	0.283	0.255	0.587	0.549	0.520
jruby-1.5.0	0.284	0.165	0.604	0.453	0.570
jruby-1.7.0.preview1	0.349	0.279	0.480	0.320	0.524
lucene-3.0.0	0.362	0.202	0.404	0.435	0.560
lucene-3.1	0.362	0.216	0.435	0.434	0.699
wicket-1.5.3	0.308	0.249	0.581	0.436	0.619
W/T/L	14/0/0	14/0/0	11/0/3	12/0/2	-
Median	0.361	0.248	0.506	0.435	0.593

Table 12 The results of Top-20 Accuracy($\uparrow$)

test_version	N.gram	ErrorProne	LineVD	DeepLineDP	LineFlowDP
activemq-5.2.0	0.451	0.403	0.625	0.470	0.612
activemq-5.3.0	0.415	0.285	0.660	0.526	0.733
activemq-5.8.0	0.496	0.390	0.594	0.585	0.725
camel-2.10.0	0.435	0.344	0.639	0.554	0.498
camel-2.11.0	0.427	0.357	0.521	0.561	0.547
derby-10.5.1.1	0.533	0.307	0.758	0.544	0.679
groovy-1_6_BETA_2	0.440	0.290	0.610	0.614	0.718
hbase-0.95.2	0.465	0.304	0.626	0.350	0.695
hive-0.12.0	0.346	0.314	0.682	0.718	0.602
jruby-1.5.0	0.347	0.232	0.736	0.646	0.670
jruby-1.7.0.preview1	0.469	0.315	0.618	0.389	0.592
lucene-3.0.0	0.450	0.246	0.566	0.556	0.655
lucene-3.1	0.422	0.263	0.551	0.632	0.761
wicket-1.5.3	0.415	0.317	0.700	0.611	0.702
W/T/L	14/0/0	14/0/0	8/0/6	11/0/3	-
Median	0.438	0.310	0.625	0.558	0.675

Detailed Results for RQ3

Table 13 Comparison table of Recall@Top 20% LOC ablation experiment (↑)

test_version	Base	Context	Flow
activemq-5.2.0	0.160	0.391	0.565
activemq-5.3.0	0.188	0.334	0.502
activemq-5.8.0	0.292	0.299	0.470
camel-2.10.0	0.062	0.117	0.343
camel-2.11.0	0.183	0.155	0.422
derby-10.5.1.1	0.266	0.189	0.390
groovy-1_6_BETA_2	0.298	0.332	0.407
hbase-0.95.2	0.087	0.457	0.678
hive-0.12.0	0.219	0.410	0.152
jruby-1.5.0	0.049	0.118	0.264
jruby-1.7.0.preview1	0.087	0.410	0.328
lucene-3.0.0	0.264	0.122	0.471
lucene-3.1	0.004	0.336	0.352
wicket-1.5.3	0.023	0.068	0.150
W/T/L	13/0/1	12/0/2	-
Median	0.171	0.316	0.398

Table 14 Comparison table of Effort@Top20%Recall ablation experiment ($\downarrow$)

test_version	Base	Context	Flow
activemq-5.2.0	0.201	0.133	0.029
activemq-5.3.0	0.255	0.232	0.092
activemq-5.8.0	0.119	0.249	0.182
camel-2.10.0	0.333	0.222	0.164
camel-2.11.0	0.436	0.301	0.050
derby-10.5.1.1	0.358	0.254	0.131
groovy-1_6_BETA_2	0.438	0.365	0.051
hbase-0.95.2	0.454	0.483	0.140
hive-0.12.0	0.290	0.120	0.292
jruby-1.5.0	0.437	0.421	0.253
jruby-1.7.0.preview1	0.143	0.380	0.134
lucene-3.0.0	0.425	0.129	0.073
lucene-3.1	0.160	0.220	0.072
wicket-1.5.3	0.213	0.234	0.131
W/T/L	12/0/2	13/0/1	-
Median	0.311	0.241	0.131

Table 15 Comparison table of IFA ablation experiment (↓)

test_version	Base	Context	Flow
activemq-5.2.0	117	90	96
activemq-5.3.0	68	75	37
activemq-5.8.0	69	46	71
camel-2.10.0	117	156	99
camel-2.11.0	100	198	268
derby-10.5.1.1	60	76	102
groovy-1_6_BETA_2	237	96	90
hbase-0.95.2	84	238	126
hive-0.12.0	169	147	95
jruby-1.5.0	167	256	62
jruby-1.7.0.preview1	334	340	62
lucene-3.0.0	212	79	99
lucene-3.1	77	64	90
wicket-1.5.3	229	47	148
W/T/L	9/0/5	7/0/7	-
Median	117	93	96

Table 16 Comparison table of Recall@Top20%LOC ablation experiment ($\uparrow$)

test_version	D	K	C	DKC
activemq-5.2.0	0.535	0.543	0.432	0.565
activemq-5.3.0	0.469	0.502	0.444	0.502
activemq-5.8.0	0.225	0.496	0.293	0.470
camel-2.10.0	0.162	0.098	0.398	0.343
camel-2.11.0	0.356	0.422	0.156	0.422
derby-10.5.1.1	0.356	0.386	0.285	0.390
groovy-1_6_BETA_2	0.387	0.362	0.359	0.407
hbase-0.95.2	0.155	0.236	0.678	0.678
hive-0.12.0	0.145	0.133	0.158	0.152
jruby-1.5.0	0.128	0.264	0.159	0.264
jruby-1.7.0.preview1	0.315	0.359	0.318	0.328
lucene-3.0.0	0.257	0.378	0.146	0.471
lucene-3.1	0.228	0.349	0.189	0.352
wicket-1.5.3	0.150	0.150	0.127	0.150
W/T/L	13/1/0	8/4/2	11/1/2	-
Median	0.242	0.360	0.289	0.398

Table 17 Comparison table of Effort@Top20%Recall ablation experiment (↓)

test_version	D	K	C	DKC
activemq-5.2.0	0.034	0.024	0.055	0.029
activemq-5.3.0	0.085	0.114	0.076	0.092
activemq-5.8.0	0.169	0.195	0.201	0.182
camel-2.10.0	0.248	0.259	0.149	0.164
camel-2.11.0	0.247	0.038	0.256	0.050
derby-10.5.1.1	0.279	0.165	0.173	0.131
groovy-1_6_BETA_2	0.048	0.046	0.068	0.051
hbase-0.95.2	0.486	0.138	0.568	0.140
hive-0.12.0	0.315	0.292	0.356	0.292
jruby-1.5.0	0.269	0.236	0.318	0.253
jruby-1.7.0.preview1	0.229	0.216	0.247	0.134
lucene-3.0.0	0.134	0.136	0.096	0.073
lucene-3.1	0.157	0.105	0.139	0.072
wicket-1.5.3	0.125	0.126	0.225	0.131
W/T/L	10/0/4	7/1/6	12/0/2	-
Median	0.199	0.137	0.187	0.131

Table 18 Comparison table of Initial False Alarms ablation experiment (↓)

test_version	D	K	C	DKC
activemq-5.2.0	98	102	92	96
activemq-5.3.0	31	45	52	37
activemq-5.8.0	62	58	86	71
camel-2.10.0	156	102	92	99
camel-2.11.0	252	275	243	268
derby-10.5.1.1	86	123	86	102
groovy-1_6_BETA_2	98	75	124	90
hbase-0.95.2	129	133	134	126
hive-0.12.0	90	96	86	95
jrubby-1.5.0	223	92	189	62
jrubby-1.7.0.preview1	104	124	87	62
lucene-3.0.0	105	126	86	99
lucene-3.1	131	124	93	90
wicket-1.5.3	124	135	129	148
W/T/L	8/0/6	11/0/3	7/0/7	-
Median	105	113	92	96

Detailed Results for RQ4

Table 19 Time Efficiency

		BoW	DBN	CNN	Bi-LSTM	CodeT5-base	DeepBugs	N-gram	ErrorProne	IVDetect	LineVD	DeepLineDP	LineFlowDP
activemq	Training	3.2	2927.8	106.3	165.9	281.8	330.8	228.5	N/A	340.3	238.9	1216.9	532.6
	Test	1.1	51.5	3.1	4.4	6.0	3.2	0.9	90.4	10.1	7.7	23.0	13.3
camel	Training	4.8	2453.6	74.4	179.4	366.4	333.1	213.3	N/A	243.2	214.6	926.9	400.5
	Test	2.5	49.8	2.3	7.1	12.6	2.9	2.0	136.6	13.2	13.7	33.8	23.3
derby	Training	3.0	2886.6	102.1	140.5	468.7	359.9	181.7	N/A	363.4	214.7	1389.0	414.6
	Test	3.6	62.0	8.3	10.3	13.9	1.7	2.5	222.7	2.2	12.8	27.4	10.6
groovy	Training	6.5	2759.4	85.6	240.3	290.6	349.5	197.2	N/A	191.1	345.8	932.0	591.3
	Test	2.6	51.0	6.1	8.4	5.1	4.1	1.7	91.1	4.6	9.4	26.4	14.7
hbase	Training	7.7	2474.0	87.3	239.6	545.2	133.0	207.7	N/A	210.7	229.5	1304.6	671.8
	Test	2.4	53.6	5.1	5.8	5.1	4.0	2.1	227.5	8.1	10.8	29.5	12.4
hive	Training	2.2	2731.6	78.8	144.8	468.6	352.1	224.9	N/A	238.0	206.3	1207.2	701.5
	Test	6.8	67.1	1.4	4.8	12.0	2.5	0.4	182.0	4.4	5.0	16.9	20.2
jruby	Training	5.0	3243.5	81.3	179.2	496.5	274.9	173.5	N/A	191.9	179.6	1039.6	472.6
	Test	5.5	29.3	4.7	10.6	5.6	1.8	0.2	224.8	4.0	7.3	15.0	11.7
lucene	Training	5.5	2710.2	112.2	212.9	361.5	303.9	194.8	N/A	202.3	248.8	1172.6	676.7
	Test	3.1	48.8	6.3	5.6	6.8	0.1	3.6	72.3	5.5	5.9	22.7	13.5
wicket	Training	3.6	2577.2	108.6	246.1	516.3	261.7	195.8	N/A	331.3	311.9	1121.2	613.8
	Test	2.7	53.7	2.4	11.8	6.3	3.7	1.8	267.9	2.0	8.5	25.1	12.5
Median	Training	4.8	2731.6	87.3	179.4	468.6	330.8	197.2	N/A	238.0	229.5	1172.6	591.3
	Test	2.7	51.5	4.7	7.1	6.3	2.9	1.8	182.0	4.6	8.5	25.1	13.3

Acknowledgements This paper is supported by the Jiangxi Provincial Key R&D Program Project(20202BBEL53002).

Author Contributions All authors are contributed equally.

Funding This paper is supported by the Jiangxi Provincial Key R&D Program Project(20202BBEL53002).

Data availability The dataset used in this paper at <https://github.com/aws-m-research/line-level-defect-prediction>. To foster the replication of our study, we will publish the implementation of our LineFlowDP and the baselines at <https://github.com/LineFlowDP/LineFlowDP>.

Declarations

Competing interests The authors declare they have no conflicts of interest.

References

- Abdous S, Abdollahzadeh R, Rohban M H (2023) KS-GNNExplainer: Global Model Interpretation Through Instance Explanations On Histopathology images[J]. arXiv preprint arXiv:2304.08240. <https://doi.org/10.48550/arXiv.2304.08240>
- Aftandilian E, Sauciu R, Priya S et al (2012) Building Useful Program Analysis Tools using An Extensible Java Compiler[C]//2012 IEEE 12th International Working Conference on Source Code Analysis and Manipulation. IEEE:14–23. <https://doi.org/10.1109/SCAM.2012.28>
- Allamanis M, Brockschmidt M, Khademi M (2017) Learning to Represent Programs with Graphs[J]. arXiv preprint arXiv:1711.00740. <https://doi.org/10.48550/arXiv.1711.00740>
- Cao S, Sun X, Bo L, et al. (2022) MVD: Memory-Related Vulnerability Detection Based on Flow-Sensitive Graph Neural Networks[J]. arXiv preprint arXiv:2203.02660. <https://doi.org/10.1145/3510003.3510219>
- Chawla NV, Bowyer KW, Hall LO et al (2002) SMOTE: Synthetic Minority Over-sampling Technique[J]. J Artif Intell Res 16:321–357. <https://doi.org/10.1613/jair.953>
- Chen X, Zhao YQ, Gu Q, Ni C, Wang Z (2019) Empirical Studies on Multi-objective File-level Software Defect Prediction Method. Ruan Jian Xue Bao/J Software 30(12):3694–3713 (in Chinese). <https://doi.org/10.13328/j.cnki.jos.005604>
- Cheng X, Wang H, Hua J et al (2021) Deepwukong: Statically Detecting Software Vulnerabilities using Deep Graph Neural Network[J]. ACM Trans Software Engin Methodology (TOSEM) 30(3):1–33. <https://doi.org/10.1145/3436877>
- Cho K, Van Merriënboer B, Gulcehre C, et al. (2014) Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation[J]. arXiv preprint arXiv:1406.1078. <https://doi.org/10.48550/arXiv.1406.1078>
- Cohen J (2013) Statistical Power Analysis for the Behavioral Sciences[M]. Academic press
- Dam HK, Tran T, Pham T et al (2018) Automatic Feature Learning for Predicting Vulnerable Software Components[J]. IEEE Trans Softw Eng 47(1):67–85. <https://doi.org/10.1109/TSE.2018.2881961>
- Ferrante J, Ottenstein K J, Warren JD . The Program Dependence Graph and Its Use in Optimization[J]. International Symposium on Programming, 6th Colloquium, Toulouse, April 17-19, 1984, Proceedings, 1984. <https://doi.org/10.1145/24039.24041>
- Freeman LC (1978) Centrality in Social Networks Conceptual Clarification[J]. Soc Networks 1(3):215–239. [https://doi.org/10.1016/0378-8733\(78\)90021-7](https://doi.org/10.1016/0378-8733(78)90021-7)
- Hata H, Mizuno O, Kikuno T (2010) Fault-prone Module Detection using Large-scale Text Features based on Spam Filtering[J]. Empir Softw Eng 15(2):147–165. <https://doi.org/10.1007/s10664-009-9117-9>
- Hellendoorn VJ, Devanbu P (2017) Are Deep Neural Networks the Best Choice for Modeling Source Code?[C]//Proceedings of the 2017 11th Joint Meeting on Foundations of. Softw Eng:763–773. <https://doi.org/10.1145/3106237.3106290>
- Hin D, Kan A, Chen H, et al. (2022) LineVD: Statement-level Vulnerability Detection using Graph Neural Networks[C]//Proceedings of the 19th International Conference on Mining Software Repositories. 596–607. <https://doi.org/10.1145/3524842.3527949>

- Hindle A, Godfrey MW, Holt RC (2008) Reading Beside the Lines: Indentation as A Proxy for Complexity Metric[C]//2008 16th IEEE International Conference on Program Comprehension. IEEE:133–142. <https://doi.org/10.1109/ICPC.2008.13>
- Huang Q, Xia X, Lo D (2017) Supervised vs unsupervised Models: A Holistic Look at Effort-aware Just-in-time Defect Prediction[C]//2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE:159–170. <https://doi.org/10.1109/ICSME.2017.51>
- Ieee S (1994) IEEE Standard Classification for Software Anomalies.[J]. IEEE Standard Indus 9(2):1–4
- Jian L, He P, Zhu J et al (2017) Software Defect Prediction via Convolutional Neural Network[C]//In: 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS). IEEE. <https://doi.org/10.1109/QRS.2017.42>
- Kamei Y, Shihab E (2016) Defect Prediction: Accomplishments and Future Challenges. Leaders Tomorrow Symposium: Future Software Engineering FOSE@SANER 2016:33–45. <https://doi.org/10.1109/SANER.2016.56>
- Katz L (1953) A New Status Index Derived From Sociometric Analysis[J]. Psychometrika 18(1):39–43. <https://doi.org/10.1007/BF02289026>
- Khakhar P, Dubey RK (2022) The Integrity of Machine Learning Algorithms Against Software Defect Prediction[M]//Artificial Intelligence and Machine Learning for EDGE Computing. Academic Press:65–74. <https://doi.org/10.1016/B978-0-12-824054-0.00027-7>
- Kingma DP, Adam BJ (2014) A Method for Stochastic Optimization[J]. arXiv preprint arXiv:1412.6980. <https://doi.org/10.48550/arXiv.1412.6980>
- Kondo M, German DM, Mizuno O et al (2020) The Impact of Context Metrics on Just-In-Time Defect Prediction[J]. Empir Softw Eng 25:890–939. <https://doi.org/10.1007/s10664-019-09736-3>
- Korel B (1987) The Program Dependence Graph in Static Program Testing[J]. Inf Process Lett 24(2):103–108. [https://doi.org/10.1016/0020-0190\(87\)90102-5](https://doi.org/10.1016/0020-0190(87)90102-5)
- Le Q, Mikolov T (2014) Distributed Representations of Sentences and Documents[C]//International conference on machine learning. PMLR:1188–1196 10.48550/ arXiv.1405.4053
- Li XZ, Qing DJ, He YP, Ma HT (2022) Fine-grained Bug Location Method Based on Source Code Extension Information. Ruan Jian Xue Bao/J Software 33(11):4008–4026 (in Chinese). <https://doi.org/10.13328/j.cnki.jos.006339>
- Li Y, Wang S, Nguyen T N (2021) Vulnerability Detection with Fine-grained Interpretations[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 292–303. <https://doi.org/10.1145/3468264.3468597>
- Lou Y, Zhu Q, Dong J, et al. (2021) Boosting Coverage-based Fault Localization via Graph-based Representation Learning[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 664–676. <https://doi.org/10.1145/3468264.3468580>
- Lucic A, Ter Hoeve MA, Tolomei G et al (2022) Cf-gnnexplainer: Counterfactual Explanations for Graph Neural Networks[C]//International Conference on Artificial Intelligence and Statistics. PMLR:4499–4511
- Lundberg S M, Lee S I (2017) A Unified Approach to Interpreting Model Predictions[J]. Advances in Neural Information Processing Systems, 30.
- Luo D, Cheng W, Xu D, et al. (2020) Parameterized Explainer for Graph Geural Getwork[J]. Advances in Neural Information Processing Systems, 33: 19620–19631. <https://doi.org/10.48550/arXiv.2011.04573>
- Miletić M, Vukušić M, Mauša G, et al. Cross-release Code Churn Impact on Effort-aware Software Defect Prediction[C]//2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO) IEEE, 2018: 1460–1466. <https://doi.org/10.23919/MIPRO.2018.8400263>
- Morasca S, Lavazza L (2020) On the Assessment of Software Defect Prediction Models via ROC Curves[J]. Empir Softw Eng 25:3977–4019. <https://doi.org/10.1007/s10664-020-09861-4>
- Nguyen HH et al (2022) MANDO: Multi-Level Heterogeneous Graph Embeddings for Fine-Grained Detection of Smart Contract Vulnerabilities. In: 2022 IEEE 9th International Conference on Data Science and Advanced Analytics (DSAA), Shenzhen, China, pp 1–10. <https://doi.org/10.1109/DSAA54385.2022.10032337>
- Parnin C, Orso A (2011) Are Automated Debugging Techniques Actually Helping Programmers?[C] //Proceedings of the 2011 international symposium on software testing and analysis. 199–209. <https://doi.org/10.1145/2001420.2001445>
- Pornprasit C, Tantithamthavorn C (2022) DeepLineDP: Towards A Deep Learning Approach for Line-Level Defect Prediction[J]. IEEE Trans Softw Eng. <https://doi.org/10.1109/TSE.2022.3144348>

- Pornprasit C, Tantithamthavorn CK (2021) JITLine: A Simpler, Better, Faster, Finer-grained Just-in-time Defect Prediction[C]//2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR). IEEE:369–379. <https://doi.org/10.1109/MSR52588.2021.00049>
- Pradel M, Sen K (2018) Deepbugs: A Learning Approach to Name-based Bug Detection[J]. *Proc ACM Prog Lang* 2(OOPSLA):1–25. <https://doi.org/10.1145/3276517>
- Rahman F, Posnett D, Devanbu P (2012) Recalling the “Imprecision” of Cross-project Defect Prediction[C]//Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering. 1–11. <https://doi.org/10.1145/2393596.2393669>
- Ray B, Hellendoorn V, Godhane S, et al. (2016) On the “Naturalness” of Buggy Code[C]//Proceedings of the 38th International Conference on Software Engineering. 428–439. <https://doi.org/10.1145/2884781.2884848>
- Ribeiro MT, Singh S, Guestrin C (2016) “Why Should I Trust You?” Explaining the predictions of any classifier[C]//Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. 1135–1144. <https://doi.org/10.1145/2939672.2939778>
- Robbins H, Monro S (1951) A Stochastic Approximation Method[J]. *Ann Math Stat* 22(3):400–407. <https://doi.org/10.1214/aoms/1177729586>
- Schlichtkrull M, Kipf T N, Bloem P, et al. Modeling Relational Data with Graph Convolutional Networks[C]//The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings 15. Springer International Publishing, 2018: 593–607. https://doi.org/10.1007/978-3-319-93417-4_38
- Sohn J, Kamei Y, McIntosh S et al (2021) Leveraging Fault Localisation to Enhance Defect Prediction[C]//2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE:284–294. <https://doi.org/10.1109/SANER50967.2021.00034>
- Staniak M, Biecek P (2018) Explanations of Model Predictions with Live and BreakDown Packages[J]. arXiv preprint arXiv:1804.01955. 10.48550/arXiv.1804.01955
- Tang L, Tao C, Guo H et al (2022) Software Defect Prediction via GCN based on Structural and Context Information[C]//2022 9th International Conference on Dependable Systems and Their Applications (DSA). IEEE:310–319. <https://doi.org/10.1109/DSA56465.2022.00049>
- Tantithamthavorn C, Hassan AE, Matsumoto K (2018b) The Impact of Class Rebalancing Techniques on the Performance and Interpretation of Defect Prediction Models[J]. *IEEE Trans Softw Eng* 46(11):1200–1219. <https://doi.org/10.1109/TSE.2018.2876537>
- Tantithamthavorn C, McIntosh S, Hassan A E, et al. (2016a) Automated Parameter Optimization of Classification Techniques for Defect Prediction Models[C]//Proceedings of the 38th International Conference on Software Engineering. 321–332. <https://doi.org/10.1145/2884781.2884857>
- Tantithamthavorn C, McIntosh S, Hassan AE et al (2016b) An Empirical Comparison of Model Validation Techniques for Defect Prediction Models[J]. *IEEE Trans Softw Eng* 43(1):1–18. <https://doi.org/10.1109/TSE.2016.2584050>
- Tantithamthavorn C, McIntosh S, Hassan AE et al (2018a) The Impact of Automated Parameter Optimization on Defect Prediction Models[J]. *IEEE Trans Softw Eng* 45(7):683–711. <https://doi.org/10.1109/TSE.2018.2794977>
- Uddin S, Hossain L, Wigand RT (2014) New Direction in Degree Centrality Measure: Towards a Time-variant Approach[J]. *Int J Inf Technol Decis Mak* 13(04):865–878. <https://doi.org/10.1109/SANER.2016.56>
- Vaswani A, Shazeer N, Parmar N, et al. (2017) Attention is All You Need[J]. *Advances in neural information processing systems*, 30. <https://doi.org/10.48550/arXiv.1706.03762>
- Wan Z , Xia X , Hassan A E , et al. (2018a) Perceptions, Expectations, and Challenges in Defect Prediction[J]. *IEEE Transactions on Software Engineering*, PP:1–1. <https://doi.org/10.1109/TSE.2018.2877678>
- Wan Z, Xia X, Hassan AE et al (2018b) Perceptions, Expectations, and Challenges in Defect Prediction[J]. *IEEE Trans Softw Eng* 46(11):1241–1266. <https://doi.org/10.1109/TSE.2018.2877678>
- Wang H, Khoshgoftaar TM, Napolitano A (2010) A comparative Study of Ensemble Feature Selection Techniques for Software Defect Prediction[C]//2010 Ninth International Conference on Machine Learning and Applications. IEEE:135–140. <https://doi.org/10.1109/ICMLA.2010.27>
- Wang S, Chollak D, Movshovitz-Attias D, et al. (2016) Bugram: Bug Detection with N-gram Language Models[C]//Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. 708–719. <https://doi.org/10.1145/2970276.2970341>
- Wang S, Liu T, Nam J et al (2018) Deep Semantic Feature Learning for Software Defect Prediction[J]. *IEEE Trans Softw Eng* 46(12):1267–1293. <https://doi.org/10.1109/TSE.2018.2877612>

- Wang W, Li G, Ma B et al (2020) Detecting Code Clones with Graph Neural Network and Flow-augmented Abstract Syntax Tree[C]//2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE:261–271. <https://doi.org/10.1109/SANER48275.2020.9054857>
- Wang Y, Wang W, Joty S, et al. (2021) Codet5: Identifier-aware Unified Pre-trained Encoder-decoder Models for Code Understanding and Generation[J]. arXiv preprint arXiv:2109.00859. <https://doi.org/10.48550/arXiv.2305.07922>
- Wattanakriengkrai S, Thongtanunam P, Tantithamthavorn C et al (2020) Predicting Defective Lines Using a Model-Agnostic Technique[J]. IEEE Transac Software Eng. <https://doi.org/10.1109/TSE.2020.3023177>
- Wong WE, Debroy V, Surampudi A et al (2010) Recent Catastrophic Accidents. In: Investigating How Software was Responsible[C]// Fourth International Conference on Secure Software Integration & Reliability Improvement. IEEE Computer Society. <https://doi.org/10.1109/TSE.2012.56>
- Wong WE, Li X, Laplante PA (2017) Be More Familiar with Our Enemies and Pave the Way Forward: A Review of the Roles Bugs Played in Software Failures[J]. J Syst Softw 133:68–94. <https://doi.org/10.1016/j.jss.2017.06.069>
- Wu B, Liang X, Zhang SS, Xu R (2022a) Advances and Applications in Graph Neural Network[J]. Chinese J Comput 45(01):35–68(in Chinese with English abstract). <https://doi.org/10.11897/SPJ.1016.2022.00035>
- Wu Y, Zou D, Dou S, et al. (2022b) VulCNN: An Image-inspired Scalable Vulnerability Detection System[C]// Proceedings of the 44th International Conference on Software Engineering. 2365–2376. <https://doi.org/10.1145/3510003.3510229>
- Wu Z, Pan S, Chen F et al (2020) A Comprehensive Survey on Graph Neural Networks[J]. IEEE Transac Neural Networks Learn Syst 32(1):4–24. <https://doi.org/10.1109/TNNLS.2020.2978386>
- Xiao Y, Jin L, Yang Z et al (2017) The Bayesian Network Based Program Dependence Graph and Its Application to Fault Localization[J]. J Syst Softw:134. <https://doi.org/10.1016/j.jss.2017.08.025>
- Xu J, Ai J, Liu J et al (2022) ACGDP: An Augmented Code Graph-Based System for Software Defect Prediction[J]. IEEE Trans Reliab. <https://doi.org/10.1109/TR.2022.3161581>
- Yan M, Xia X, Fan Y et al (2020) Just-in-time Defect Identification and Localization: A two-phase Framework[J]. IEEE Trans Softw Eng 48(1):82–101. <https://doi.org/10.1109/TSE.2020.2978819>
- Yang F Y, Zeng G D, Zhong F, et al. (2023) Interpretable Software Defect Prediction Incorporating Multiple Rules[C]//2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE. 940–947.
- Yang Z, Yang D, Dyer C, et al. (2016) Hierarchical Attention Networks for Document Classification[C]//Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 1480–1489. <https://doi.org/10.18653/v1/N16-1174>
- Ying R , Bourgeois D , You J , et al. (2019) GNNExplainer: Generating Explanations for Graph Neural Networks[J]. Advances in Neural Information Processing Systems, 32:9240–9251. <https://doi.org/10.48550/arXiv.1903.03894>
- Zeng C, Zhou CY, Lv SK et al (2021) GCN2defect: Graph Convolutional Networks for SMOTETomek-based Software Defect Prediction[C]//2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE). IEEE:69–79. <https://doi.org/10.1109/ISSRE52982.2021.00020>
- Zhang Z, Lei Y, Yan M, et al. (2022) Reentrancy Vulnerability Detection and Localization: A Deep Learning Based Two-phase Approach[C]//Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. 1–13. <https://doi.org/10.1145/3551349.3560428>
- Zheng W, Chen TF, Hu MT et al (2023) Hybrid Defect Prediction Model Based on Counterfactual Feature Optimization[J]. Human-Centric Intel Syst:1–15
- Zhu Q (2020) On the Performance of Matthews Correlation Coefficient (MCC) for Imbalanced Dataset[J]. Pattern Recogn Lett 136:71–80. <https://doi.org/10.1016/j.patrec.2020.03.030>

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.